

Stefan Heiss

Technische Hochschule Ostwestfalen-Lippe
Fachbereich Elektrotechnik und Technische Informatik

Datensicherheit

03 – Asymmetrische Verfahren

Version vom 2. April 2025

Vorwort

Liebe Studierende,

im Studienheft 02 zum Modul Datensicherheit wurden die Hashfunktionen und die symmetrischen kryptographischen Verfahren behandelt. Zur Anwendung eines symmetrischen kryptographischen Verfahrens (symmetrisches Verschlüsselungsverfahren oder MAC-Verfahren) benötigen Alice und Bob einen gemeinsamen geheimen Schlüssel.

In diesem Studienheft 03 werden asymmetrische kryptographische Verfahren (*Public Key Verfahren*) behandelt, bei denen Schlüsselpaare zum Einsatz kommen, die aus einem *privaten* und einem *öffentlichen* Schlüsselteil bestehen. Grundlegende Mechanismen, die asymmetrische kryptographische Verfahren nutzen, sind:

- *Schlüsselaustauschverfahren*,
- *asymmetrische Verschlüsselungsverfahren* und
- *Signaturverfahren*.

Die schon klassisch zu nennenden DH- und RSA-Verfahren werden in diesem Studienheft 03 detailliert behandelt. Zum Verständnis der Algorithmen werden einige weitere Eigenschaften der endlichen Zahlbereiche $\mathbb{Z}_n$ benötigt. Diese werden daher zunächst im Abschnitt 1 bereitgestellt.

Inhaltsverzeichnis

1	Modulare Arithmetik - Teil 2	1
1.1	Potenzen	2
1.2	Mathematischer Exkurs: Einheitengruppen $\mathbb{Z}_{p^e}^*$	8
1.3	Der chinesische Restsatz	11
1.4	Mathematischer Exkurs: Elemente ungerader und gerader Ordnung in $\mathbb{Z}_n$	17
2	Das Diffie-Hellman-Schlüsselaustauschverfahren	23
2.1	Exkurs: Berechnung modularer Potenzen	30
2.2	Exkurs: Angriffe auf das DH-Verfahren	34
2.2.1	Pohlig-Hellman-Reduktion	34
2.2.2	Shanks' „Baby Steps Giant Steps“-Verfahren (BSGS-Verfahren)	35
2.2.3	Pollard's <i>Rho-Methode</i>	35
2.2.4	Zahlkörpersieb	37
3	Das RSA-Verfahren	39
3.1	RSA-Schlüsselpaare	41
3.2	Das RSA-Verschlüsselungsverfahren	45
3.3	Das RSA-Signaturverfahren	51

Modulare Arithmetik - Teil 2

1

1.1	Potenzen	2
1.2	Mathematischer Exkurs: Einheitengruppen $\mathbb{Z}_{p^e}^*$	8
1.3	Der chinesische Restsatz	11
1.4	Mathematischer Exkurs: Elemente ungerader und gerader Ordnung in $\mathbb{Z}_n$	17

1.1 Potenzen

Für beliebige Zahlen $a \in \mathbb{Z}_n$ sind Potenzen für natürliche Exponenten $i \in \mathbb{N}$ wie üblich definiert:

$$a^i := \underbrace{a \cdot a \cdots a}_i \quad \text{für } a \in \mathbb{Z}_n, i \in \mathbb{N} \quad (1.1)$$

Ist a invertierbar, d. h. $a \in \mathbb{Z}_n^*$, so sind Potenzen sogar für alle ganzzahligen Exponenten definiert, indem die Festlegung in (1.1) durch

$$a^0 := 1 \quad \text{für } a \in \mathbb{Z}_n^* \quad (1.2)$$

und

$$a^{-i} := (a^{-1})^i \quad \text{für } a \in \mathbb{Z}_n^*, i \in \mathbb{N} \quad (1.3)$$

ergänzt wird. Die bekannten Potenzgesetze gelten auch in $\mathbb{Z}_n$ bzw. $\mathbb{Z}_n^*$:

Sind $a, b \in \mathbb{Z}_n$ und $i, j \in \mathbb{N}$ oder sind $a, b \in \mathbb{Z}_n^*$ und $i, j \in \mathbb{Z}$, so gilt:

- (i) $a^i \cdot a^j = a^{i+j}$
- (ii) $(a^i)^j = a^{ij}$
- (iii) $a^i \cdot b^i = (ab)^i$

Erzeugnis eines invertierbaren Elements

Für $a \in \mathbb{Z}_n^*$ wird die Menge aller Potenzen von a als das *Erzeugnis* von a in $\mathbb{Z}_n^*$ oder auch als die von dem Element a erzeugte zyklische Untergruppe der Einheitengruppe bezeichnet. In Zeichen:

$$\langle a \rangle := \{a^i \mid i \in \mathbb{Z}\}$$

Definition

Ist $a \in \mathbb{Z}_n^*$, so wird die Anzahl der paarweise verschiedenen Potenzen von a als *Ordnung* von a in $\mathbb{Z}_n^*$ bezeichnet. In Zeichen:

$$o_n(a) := |\langle a \rangle|$$

Für die Ordnung von a in $\mathbb{Z}_n^*$ wird auch nur $o(a)$ geschrieben, falls n aus dem Zusammenhang bekannt ist.

Frage 1

Bestimmen Sie für alle Elemente aus $\mathbb{Z}_{13}^*$ die Ordnung und das jeweils inverse Element.

Satz 1.1

Ist $a \in \mathbb{Z}_n^*$, so gilt $a^{o(a)} = 1$ und:

$$\langle a \rangle = \{1, a, a^2, a^3, \dots, a^{o(a)-1}\}$$

Beweis Es sei $a \in \mathbb{Z}_n^*$ und a^i die erste Wiederholung eines Elements in der Folge der Potenzen von a mit nichtnegativen Exponenten:

$$a^0 = 1, a, a^2, a^3, \dots$$

Es sei also $i \in \mathbb{N}$ minimal mit der Eigenschaft, dass es ein $j < i$ gibt mit:

$$a^i = a^j$$

Multiplikation mit a^{-j} liefert:

$$a^{i-j} = 1$$

Wegen der minimalen Wahl von i ist folglich $j = 0$ und:

$$a^i = 1$$

Es gilt also

$$\begin{aligned} &1, a, a^2, a^3, \dots, a^{i-1}, a^i, a^{i+1}, a^{i+2}, a^{i+3}, \dots \\ &= 1, a, a^2, a^3, \dots, a^{i-1}, 1, a, a^2, a^3, \dots \end{aligned}$$

mit paarweise verschiedenen Elementen:

$$1, a, a^2, a^3, \dots, a^{i-1}$$

Für jedes $k \in \mathbb{Z}$ gilt:

$$a^k = a^{i \cdot (k/\mathbb{Z} i) + (k \bmod i)} = (a^i)^{k/\mathbb{Z} i} \cdot a^{k \bmod i} = 1^{k/\mathbb{Z} i} \cdot a^{k \bmod i} = a^{k \bmod i}$$

Somit gilt

$$\langle a \rangle = \{1, a, a^2, a^3, \dots, a^{i-1}\}$$

und:

$$a^{o(a)} = a^i = 1$$

■

Satz 1.2

Es seien $n \in \mathbb{N}$ ($n \geq 2$), $a, b \in \mathbb{Z}_n^*$ und $i \in \mathbb{Z}$. Dann gilt:

- (i) $a^i = 1 \implies o(a) \mid i$
- (ii) $o(a^i) \mid o(a)$
- (iii) $i \mid o(a) \implies o(a^i) = \frac{o(a)}{|i|}$
- (iv) $\text{ggT}(o(a), i) = 1 \implies o(a^i) = o(a)$
- (v) $o(ab) \mid o(a) \cdot o(b)$
- (vi) $\text{ggT}(o(a), o(b)) = 1 \implies o(a \cdot b) = o(a) \cdot o(b)$

Beweis

- (i) Division von
- i
- durch
- $o(a)$
- liefert
- $q \in \mathbb{Z}$
- und
- $r = i \bmod o(a)$
- mit:

$$i = o(a) \cdot q + r$$

Hiermit ergibt sich

$$1 = a^i = a^{o(a) \cdot q + r} = \left(a^{o(a)}\right)^q \cdot a^r = 1^q \cdot a^r = a^r$$

und wegen $0 \leq r < o(a)$ folgt mit Satz 1.1:

$$r = 0 \implies i = o(a) \cdot q \implies o(a) \mid i$$

- (ii)

$$(a^i)^{o(a)} = a^{i \cdot o(a)} = \left(a^{o(a)}\right)^i = 1^i = 1 \xrightarrow{(i)} o(a^i) \mid o(a)$$

- (iii) Da
- $e = o(a)$
- die kleinste natürliche Zahl mit
- $a^e = 1$
- ist, ist
- $j = \frac{o(a)}{|i|}$
- die kleinste natürliche Zahl mit:

$$(a^i)^j = 1$$

- (iv) Wir zeigen zuerst, dass unter der Voraussetzung
- $\text{ggT}(o(a), i) = 1$
- die Zahl
- a
- eine Potenz von
- a^i
- ist. Der erweiterte Euklid'sche Algorithmus liefert nämlich ganze Zahlen
- j, k
- mit

$$i \cdot j + o(a) \cdot k = 1$$

und es folgt:

$$a = a^{i \cdot j + o(a) \cdot k} = (a^i)^j \cdot \left(a^{o(a)}\right)^k = (a^i)^j$$

Mit (ii) folgt daher nicht nur $o(a^i) \mid o(a)$ sondern auch $o(a) \mid o(a^i)$. Zusammen ergibt dies $o(a^i) = o(a)$.

- (v)

$$(ab)^{o(a) \cdot o(b)} = \left(a^{o(a)}\right)^{o(b)} \cdot \left(b^{o(b)}\right)^{o(a)} = 1 \xrightarrow{(i)} o(ab) \mid o(a) \cdot o(b)$$

- (vi) Unter der Voraussetzung
- $\text{ggT}(o(a), o(b)) = 1$
- ist zu zeigen, dass sowohl
- $o(a)$
- als auch
- $o(b)$
- Teiler von
- $o(ab)$
- sind, denn dann gilt wegen der Teilerfremdheit von
- $o(a)$
- und
- $o(b)$

$$o(a) \cdot o(b) \mid o(ab)$$

und zusammen mit (v) folgt die Behauptung.

Zum Beweis von $o(a) \mid o(ab)$: Allgemein gilt

$$1 = \left((ab)^{o(ab)}\right)^{o(b)} = a^{o(b) \cdot o(ab)} \cdot b^{o(b) \cdot o(ab)} = \left(a^{o(b)}\right)^{o(ab)} \xrightarrow{(i)} o\left(a^{o(b)}\right) \mid o(ab)$$

und wegen $\text{ggT}(o(a), o(b)) = 1$ folgt weiterhin:

$$o(a) \stackrel{(iv)}{=} o\left(a^{o(b)}\right) \mid o(ab)$$

Ebenso kann $o(b) \mid o(ab)$ geschlossen werden und (vi) ist damit vollständig bewiesen. ■

Satz 1.3

Für $n \in \mathbb{N}$, $n \geq 2$ bezeichne μ_n die maximale Ordnung eines Elementes aus $\mathbb{Z}_n^*$:

$$\mu_n := \max\{o(a) \mid a \in \mathbb{Z}_n^*\}$$

Dann ist die Ordnung von jedem Element aus $\mathbb{Z}_n^*$ ein Teiler von μ_n :

$$o(a) \mid \mu_n \quad \text{für alle } a \in \mathbb{Z}_n^*$$

Beweis Sei $m \in \mathbb{Z}_n^*$ mit $o(m) = \mu_n$. Angenommen es gibt ein $a \in \mathbb{Z}_n^*$ mit $o(a) \nmid \mu_n$.

Dann gibt es eine Primzahl p , so dass die maximale Primzahlpotenz p^s , die $o(a)$ teilt, kein Teiler von μ_n ist. Es gilt dann:

$$\begin{aligned} o(m) &= p^r \cdot q & \text{mit } \text{ggT}(p, q) &= 1 \\ o(a) &= p^s \cdot q' & \text{mit } \text{ggT}(p, q') &= 1 \text{ und } s > r \end{aligned}$$

Für

$$b := a^{q'} \cdot m^{p^r}$$

gilt mit Satz 1.2 (iii) und (vi)

$$o(b) := p^s \cdot q > p^r \cdot q = \mu_n$$

im Widerspruch zur Definition von μ_n ! ■

Für den Fall, dass $n = p$ eine Primzahl ist und $\mathbb{Z}_p$ dann einen Körper bildet, lassen sich immer sogenannte *primitive* Elemente finden, deren Potenzen alle Elemente $\neq 0$ erzeugen. Dieses wichtige Ergebnis soll im Folgenden begründet werden.

Hierfür sei zunächst an den bekannten Zusammenhang zwischen Nullstellen und Linearfaktoren von Polynomen erinnert. Für Polynome über $\mathbb{Z}_n$ ($n \in \mathbb{N}$) lautet dieser:

Satz 1.4

Es sei $n \in \mathbb{N}$ und $p(x)$ ein Polynom in $\mathbb{Z}_n[x]$ vom Grad k . Ist $a \in \mathbb{Z}_n$ eine Nullstelle von $p(x)$, so gibt es ein Polynom $q(x) \in \mathbb{Z}_n[x]$ vom Grad $k-1$ mit:

$$p(x) = (x-a) \cdot q(x)$$

Beweis Eine Polynomdivision von $p(x)$ durch $x-a$ liefert ein Polynom $q(x)$ und eine Zahl $r \in \mathbb{Z}_n$ mit:

$$p(x) = (x-a) \cdot q(x) + r$$

Ist $a \in \mathbb{Z}_n$ eine Nullstelle von $p(x)$ so folgt weiter:

$$0 = p(a) = 0 \cdot q(a) + r = r$$

■

Kommentar Bei der Division beliebiger Polynome durch ein gegebenes Polynom $b(x) \in \mathbb{Z}_n[x]$ sollte für den sich jeweils ergebenden Rest $r(x) \in \mathbb{Z}_n[x]$ immer $\text{grad}(r) < \text{grad}(b)$ gelten. Aus diesem Grund ist ein Polynom $b(x)$ nur dann als Divisor geeignet, falls der Koeffizient

vor der höchsten in $b(x)$ vorkommenden Potenz von x invertierbar ist. (Dies gilt insbesondere für *normierte* Polynome, bei denen dieser Koeffizient per Definition 1 ist.)

Beispielsweise kann in $\mathbb{Z}_6[x]$ das Polynom $b(x) = 3x$ nicht sinnvoll als Divisor zur Division von $a(x) = x^2 + x$ benutzt werden. ◀

Frage 2

Gegeben sei das Polynom:

$$p(x) = \sum_{i=0}^k c_i x^i$$

Zeigen Sie, dass

$$p(x) = (x-a) \cdot q(x) + r$$

genau dann gilt, falls

$$q(x) = \sum_{i=0}^{k-1} d_i x^i$$

mit

$$\begin{aligned} d_{k-1} &= c_k \\ d_{k-2} &= c_{k-1} + a \cdot d_{k-1} \\ &\vdots \\ d_i &= c_{i+1} + a \cdot d_{i+1} \\ &\vdots \\ d_0 &= c_1 + a \cdot d_1 \end{aligned}$$

und

$$r = c_0 + a \cdot d_0$$

ist.

Satz 1.5

Ist p eine Primzahl, so gibt es ein Element $g \in \mathbb{Z}_p^*$ mit $o(g) = p-1$, d. h.:

$$\mathbb{Z}_p^* = \langle g \rangle = \{1, g, g^2, \dots, g^{p-2}\}$$

Beweis Wir betrachten das Polynom:

$$m(x) = x^{\mu_p} - 1 \in \mathbb{Z}_p[x]$$

Mit Satz 1.3 ist jedes Element $i \in \mathbb{Z}_p^*$ eine Nullstelle von $m(x)$, d. h.:

$$m(i) = 0 \quad \text{für } i = 1, \dots, p-1$$

Die Faktorisierung von $m(x)$ mithilfe der Nullstelle 1 liefert ein Polynom $m_1(x) \in \mathbb{Z}_p[x]$ mit:

$$m(x) = (x-1) \cdot m_1(x) \quad \text{mit } \text{grad}(m_1) = \mu_p - 1$$

Für $i = 2$ ist $(i-1) \in \mathbb{Z}_p^*$ und mit der Kürzungsregel (Studienheft DC-01, Satz 2.5 (iii)) folgt $m_1(2) = 0$. Die Faktorisierung von $m(x)$ kann daher mithilfe der Nullstelle 2 von $m_1(x)$ fortgesetzt werden und liefert ein Polynom $m_2(x) \in \mathbb{Z}_p[x]$ mit:

$$m(x) = (x-1) \cdot (x-2) \cdot m_2(x) \quad \text{mit } \text{grad}(m_2) = \mu_p - 2$$

Für $i = 3$ ist $(i-1)(i-2) \in \mathbb{Z}_p^*$ und es gibt daher ein Polynom $m_3(x) \in \mathbb{Z}_p[x]$ mit:

$$m(x) = (x-1) \cdot (x-2) \cdot (x-3) \cdot m_3(x) \quad \text{mit } \text{grad}(m_3) = \mu_p - 3$$

Führen wir diese Faktorisierung fort, so ergibt sich eine vollständige Zerlegung von $m(x)$ in μ_p viele Linearfaktoren:

$$m(x) = (x-1) \cdot (x-2) \cdots (x-\mu_p)$$

Wäre $\mu_p < p-1$ so folgte:

$$0 = m(p-1) = (p-2) \cdot (p-3) \cdots (p-1-\mu_p) \in \mathbb{Z}_p^*$$

Dieser Widerspruch zeigt, dass $\mu_p = p-1$ gelten muss. Daher gibt es ein Element $g \in \mathbb{Z}_p^*$ mit:

$$o(g) = \mu_p = p-1$$

■

Primitive Elemente in $\mathbb{Z}_p$

Ist p eine Primzahl, so werden die nach Satz 1.5 existierenden Elemente $g \in \mathbb{Z}_p^*$ mit $o(g) = p-1$ als *primitive* Elemente bezeichnet.

Frage 3

Welche Elemente in $\mathbb{Z}_{13}^*$ sind primitiv?

Frage 4

Bestimmen Sie für $p \in \{2, 3, 5, 7, 11, 13, 17, 19, 23\}$ jeweils das kleinste primitive Element in $\mathbb{Z}_p^*$.

Wichtig Ist p eine Primzahl und g ein primitives Element in $\mathbb{Z}_p$, so lässt sich jedes Element $a \in \mathbb{Z}_p^*$ als Potenz von g darstellen, d. h. es gibt ein $i \in \mathbb{N}_0$ mit

$$a = g^i$$

und es gilt daher:

$$a^{p-1} = (g^i)^{p-1} = (g^{p-1})^i = 1 \quad \text{für alle } a \in \mathbb{Z}_p^* \quad (1.4)$$

Mit Satz 1.2 (i) folgt:

$$o(a) \mid (p-1) \quad \text{für alle } a \in \mathbb{Z}_p^* \quad (1.5)$$

(1.5) folgt zwar auch mit Satz 1.3 wegen $\mu_p = p-1$, aber es schadet sicherlich nichts, sich die obige Begründung zu merken. ◀

Die Gültigkeit von (1.5) für alle $a \in \mathbb{Z}_p^*$ (p Primzahl), entspricht dem *Kleinen Satz von Fermat* der elementaren Zahlentheorie.

Satz 1.6 (Kleiner Satz von Fermat)

Ist p eine Primzahl, so gilt für alle $a \in \mathbb{Z}$ mit $\text{ggT}(a, p) = 1$:

$$a^{p-1} \equiv 1 \pmod{p}$$

Beweis

$$a^{p-1} \bmod p = (a \bmod p)^{p-1} \bmod p \stackrel{(1.5)}{=} 1$$

■

Frage 5

Es sei p eine Primzahl. Zeigen Sie, dass dann gilt:

$$(i) \quad a^p = a \quad \text{für alle } a \in \mathbb{Z}_p \quad (1.6)$$

$$(ii) \quad a^p \equiv a \pmod{p} \quad \text{für alle } a \in \mathbb{Z}$$

Frage 6

Zeigen Sie, dass Satz 1.5 nicht für alle $\mathbb{Z}_n$ gilt. Finden Sie hierzu die drei kleinsten $n \in \mathbb{N}$ ($n \geq 2$) mit $\mu_n < |\mathbb{Z}_n^*|$.

Da Satz 1.5 für zusammengesetzte natürliche Zahlen im allgemeinen nicht stimmt, muss im Beweis des Satzes offenbar eine besondere Eigenschaft von Primzahlen Eingang gefunden haben. Können Sie diese Eigenschaft identifizieren? Spätestens nach der Bearbeitung der folgenden Frage, sollte dies keine Schwierigkeiten machen.

Frage 7

Finden Sie in $\mathbb{Z}_8$ alle Nullstellen von $p(x) = x^2 - 1 \in \mathbb{Z}_8[x]$ und alle Faktorisierungen von $p(x)$ in Linearfaktoren.

1.2 Mathematischer Exkurs: Einheitengruppen $\mathbb{Z}_{p^e}^*$

Ist p eine Primzahl, so ist die Einheitengruppe von $\mathbb{Z}_p$ nach Satz 1.5 *zyklisch*, d. h. es gibt primitive Elemente $g \in \mathbb{Z}_p^*$, durch dessen Potenzen alle Elemente von $\mathbb{Z}_p^*$ *generiert* werden können:

$$\langle g \rangle = \mathbb{Z}_p^*$$

Ein primitives Element wird daher auch als *Generator* von $\mathbb{Z}_p^*$ bezeichnet.

In diesem Abschnitt sollen die Einheitengruppen von $\mathbb{Z}_{p^e}$ für Primzahlpotenzen mit $e > 1$ untersucht werden. Mit den Ergebnissen dieses Abschnitts lässt sich zusammen mit dem chinesischen Restsatz in der durch Satz 1.10 in formulierten Form die Struktur der Einheitengruppen $\mathbb{Z}_n^*$ für alle $n \in \mathbb{N}$ beschreiben.

Satz 1.7

Ist p eine ungerade Primzahl und $e \in \mathbb{N}$, so gibt es ein $g \in \mathbb{Z}_{p^e}^*$ mit:

$$o(g) = p^{e-1}(p-1) = \varphi(p^e) = |\mathbb{Z}_{p^e}^*|$$

D. h. $\mathbb{Z}_{p^e}^*$ ist zyklisch mit Generator g :

$$\langle g \rangle = \mathbb{Z}_{p^e}^*$$

Beweis Wegen Satz 1.2 (vi) genügt es zu zeigen, dass es in $\mathbb{Z}_{p^e}^*$ sowohl Elemente der Ordnung $p-1$ als auch Elemente der Ordnung p^{e-1} gibt.

(i) In $\mathbb{Z}_{p^e}^*$ gibt es Elemente der Ordnung $p-1$:

Wegen Satz 1.5 gibt es ein $a \in \mathbb{Z}_p^*$ mit

$$a^{p-1} \equiv 1 \pmod{p} \quad (*)$$

und:

$$a^r \not\equiv 1 \pmod{p} \quad \text{für } 1 \leq r < p-1 \quad (**)$$

Wegen $p \nmid a$ ist a auch ein Element von $\mathbb{Z}_{p^e}^*$. Bezeichnet $o(a)$ die Ordnung von a in $\mathbb{Z}_{p^e}^*$, so ergibt eine Division von $o(a)$ durch $p-1$ mit Rest den folgenden Ansatz:

$$o(a) = (p-1) \cdot v + r \quad \text{mit } 0 \leq r < p-1$$

Aus

$$a^{(p-1) \cdot v + r} \equiv 1 \pmod{p^e} \quad (***)$$

folgt dann

$$a^r \stackrel{(*)}{\equiv} (a^{p-1})^v \cdot a^r \equiv a^{(p-1) \cdot v + r} \stackrel{(***)}{\equiv} 1 \pmod{p}$$

und daher mit (**) schließlich $r = 0$.

Die Ordnung von a in $\mathbb{Z}_{p^e}^*$ ist also ein Vielfaches von $p-1$. Eine geeignete Potenz von a besitzt daher die Ordnung $p-1$ in $\mathbb{Z}_{p^e}^*$.

(ii) In $\mathbb{Z}_{p^e}^*$ gibt es Elemente der Ordnung p^{e-1} :

Ein solches Element kann mit

$$a := 1 + p$$

konkret angegeben werden, da für $a = 1 + p$ folgende Kongruenzen gelten:

$$\begin{aligned} a &\equiv 1 + p \pmod{p^2} \\ a^p &\equiv 1 + p^2 \pmod{p^3} \\ &\vdots \quad \vdots \quad \vdots \\ a^{p^{e-2}} &\equiv 1 + p^{e-1} \pmod{p^e} \\ a^{p^{e-1}} &\equiv 1 \pmod{p^e} \end{aligned}$$

Von der Gültigkeit der obigen Kongruenzen sollten Sie sich mithilfe eines Induktionsbeweises selbständig überzeugen (s. Frage 8). ■

Frage 8

Es sei p eine ungerade Primzahl und $a := 1 + p$. Zeigen Sie per vollständiger Induktion, dass für alle $i \in \mathbb{N}$ gilt:

$$a^{p^{i-1}} \equiv 1 + p^i \pmod{p^{i+1}}$$

Kommentar Aus dem Beweis von Satz 1.7 lässt sich folgende Darstellungsmöglichkeit für die Elemente aus $\mathbb{Z}_{p^e}^*$ für ungerade Primzahlen p entnehmen:

Ist $g \in \mathbb{Z}_p$ ein primitives Element in $\mathbb{Z}_p^*$, so gilt mit $a := g^{\frac{o_{p^e}(g)}{p-1}}$:

$$\mathbb{Z}_{p^e}^* = \langle a \rangle \cdot \langle 1+p \rangle$$

$$:= \{a^i \cdot (1+p)^j \mid i = 0, 1, \dots, p-2, j = 0, 1, \dots, p^{e-1} - 1\}$$

(Zur Erinnerung: $o_{p^e}(g)$ bezeichnet die Ordnung von a in $\mathbb{Z}_{p^e}^*$.)

Beispiel 1.1

- (i) $\mathbb{Z}_2^* = \{1\}$
- (ii) $\mathbb{Z}_4^* = \{1, 3\} = \langle 3 \rangle$
- (iii) $\mathbb{Z}_8^* = \{1, 3, 5, 7\}$ ist nicht zyklisch, da $o(3) = o(5) = o(7) = 2$.
- (iv) $\mathbb{Z}_{16}^* = \{1, 3, 5, 7, 9, 11, 13, 15\}$ ist nicht zyklisch. Es gilt:

$$3^2 = 5^2 = 11^2 = 13^2 = 9, \quad 9^2 = 7^2 = 15^2 = 1$$

Bemerkenswerter Weise gibt es bezüglich der multiplikativen Struktur von $\mathbb{Z}_{16}$ keinen Unterschied zwischen den Zahlen $-1 = 15$ und 7 .

Satz 1.8

Für $e \geq 3$ ist $\mathbb{Z}_{2^e}^*$ nicht zyklisch. In $\mathbb{Z}_{2^e}^*$ gilt

$$o(5) = 2^{e-2}$$

und:

$$\mathbb{Z}_{2^e}^* = \langle 2^e - 1 \rangle \cdot \langle 5 \rangle$$

$$= \{(2^e - 1)^i \cdot 5^j \mid i = 0, 1, j = 0, 1, \dots, 2^{e-2} - 1\}$$

Beweis Besäße $\mathbb{Z}_{2^e}^*$ ein erzeugendes Element, so gäbe es auch nur ein Element der Ordnung 2 in $\mathbb{Z}_{2^e}^*$. Es gilt aber

$$o(2^e - 1) = o(2^{e-1} - 1) = o(2^{e-1} + 1) = 2$$

Mit dem Binomialsatz folgt (s. Frage 9):

$$\begin{aligned} 5 &\equiv 1 + 2^2 \pmod{2^3} \\ 5^2 &\equiv 1 + 2^3 \pmod{2^4} \\ &\vdots \quad \vdots \quad \vdots \\ 5^{2^{e-3}} &\equiv 1 + 2^{e-1} \pmod{2^e} \\ 5^{2^{e-2}} &\equiv 1 \pmod{2^e} \end{aligned}$$

Daher gilt $o(5) = 2^{e-2}$ und das (einzige) Element der Ordnung 2 in $\langle 5 \rangle$ ist $1 + 2^{e-1}$. Somit ist

$$2^e - 1 \notin \langle 5 \rangle \quad (*)$$

und für

$$(2^e - 1) \cdot \langle 5 \rangle := \{(2^e - 1) \cdot 5^j \mid j = 0, 1, \dots, 2^{e-2} - 1\}$$

gilt dann (s. Frage 10):

$$\langle 5 \rangle \cap (2^e - 1) \cdot \langle 5 \rangle = \emptyset \quad (**)$$

Folglich enthält

$$\langle 2^e - 1 \rangle \cdot \langle 5 \rangle := \{(2^e - 1)^i \cdot 5^j \mid i = 0, 1, j = 0, 1, \dots, 2^{e-2} - 1\}$$

genau $2 \cdot 2^{e-2} = |Z_{2^e}^*|$ viele Elemente, womit die Behauptung vollständig bewiesen ist. ■

Frage 9

Zeigen Sie per vollständiger Induktion, dass für alle $i \in \mathbb{N}$ gilt:

$$5^{2^{i-1}} \equiv 1 + 2^{i+1} \pmod{2^{i+2}}$$

Frage 10

Begründen Sie die Gültigkeit von (**) im Beweis von Satz 1.8.

1.3 Der chinesische Restsatz

Der chinesische Restsatz macht eine Aussage über simultane Lösungen für ein System von Kongruenzen.

Satz 1.9 (Chinesischer Restsatz)

Sind $n_1, \dots, n_r$ paarweise teilerfremde natürliche Zahlen, so besitzt für gegebene ganze Zahlen $a_1, \dots, a_r$ das System von Kongruenzen

$$\begin{aligned} x &\equiv a_1 \pmod{n_1} \\ x &\equiv a_2 \pmod{n_2} \\ &\vdots \quad \quad \quad \vdots \\ x &\equiv a_r \pmod{n_r} \end{aligned}$$

eine eindeutig bestimmte Lösung $a \in \mathbb{Z}$ mit:

$$0 \leq a < n_1 n_2 \cdots n_r$$

Die Menge aller ganzzahligen Lösungen ist durch

$$\mathbb{L} = \{a + v \cdot n_1 n_2 \cdots n_r \mid v \in \mathbb{Z}\}$$

gegeben.

Beweis Zur Konstruktion einer Lösung setzen wir

$$n := n_1 n_2 \cdots n_r$$

und für $i = 1, \dots, r$:

$$m_i := \prod_{j=1, j \neq i}^r n_j = \frac{n}{n_i}$$

Dann sind n_i und m_i für $i = 1, \dots, r$ teilerfremd und mithilfe des erweiterten Euklid'schen Algorithmus können u_i und v_i mit

$$u_i \cdot n_i + v_i \cdot m_i = 1 \quad (*)$$

berechnet werden.

Für die Zahlen

$$e_i := v_i \cdot m_i \quad (i = 1, \dots, r) \quad (**)$$

gilt wegen (*) beziehungsweise (**):

$$\begin{aligned} e_i &\equiv 1 \pmod{n_i} \\ e_i &\equiv 0 \pmod{m_i} \end{aligned}$$

Mithilfe der e_i kann nun direkt eine Lösung des Systems angegeben werden, nämlich:

$$\tilde{a} := a_1 \cdot e_1 + a_2 \cdot e_2 + \dots + a_r \cdot e_r$$

Bitte überzeugen Sie sich selbstständig davon, dass $\tilde{a}$ die r Kongruenzen aus dem Satz erfüllt.

Mit $\tilde{a}$ ist auch

$$a := \tilde{a} \bmod n$$

und jede Zahl aus

$$\{a + v \cdot n \mid v \in \mathbb{Z}\}$$

eine simultane Lösung der Kongruenzen.

Ist schließlich b eine weitere simultane Lösung der Kongruenzen, so gilt:

$$a \equiv a_i \equiv b \pmod{n_i} \quad \text{für } i = 1, \dots, r$$

Die Differenz $b - a$ wird dann von allen n_i für $i = 1, \dots, r$ geteilt. Da die n_i paarweise teilerfremd sind, wird die Differenz $b - a$ auch von dem Produkt dieser n_i , also von n , geteilt und es gibt ein $v_0 \in \mathbb{Z}$ mit:

$$b - a = v_0 \cdot n \implies b = a + v_0 \cdot n \in \{a + v \cdot n \mid v \in \mathbb{Z}\}$$

■

Beispiel 1.2

Eine Lösung für folgendes System von Kongruenzen soll berechnet werden:

$$x \equiv 2 \pmod{5}$$

$$x \equiv 7 \pmod{21}$$

$$x \equiv 6 \pmod{11}$$

Bestimmung von e_1 : Für $n_1 = 5$ und $m_1 = 21 \cdot 11 = 231$ liefert der erweiterte Euklidische Algorithmus:

$$(-46) \cdot 5 + 1 \cdot 231 = 1$$

Daher ist $e_1 = 1 \cdot 231$.

Bestimmung von e_2 : Für $n_2 = 21$ und $m_2 = 5 \cdot 11 = 55$ liefert der erweiterte Euklidische Algorithmus:

$$21 \cdot 21 + (-8) \cdot 55 = 1$$

Daher ist $e_2 = (-8) \cdot 55 = -440$.

Bestimmung von e_3 : Für $n_3 = 11$ und $m_3 = 5 \cdot 21 = 105$ liefert der erweiterte Euklidische Algorithmus:

$$(-19) \cdot 11 + 2 \cdot 105 = 1$$

Daher ist $e_3 = 2 \cdot 105 = 210$.

Die gesuchte Lösung berechnet sich nun mit $n = 5 \cdot 21 \cdot 11 = 1155$ zu:

$$\begin{aligned} a &= (2 \cdot e_1 + 7 \cdot e_2 + 6 \cdot e_3) \bmod 1155 \\ &= (2 \cdot 231 - 7 \cdot 440 + 6 \cdot 210) \bmod 1155 \\ &= -1358 \bmod 1155 \\ &= 952 \end{aligned}$$

Strukturell liegt dem chinesischen Restsatz eine Isomorphie zwischen den Ringen

$$\mathbb{Z}_{n_1 n_2 \dots n_r}$$

und

$$\mathbb{Z}_{n_1} \times \mathbb{Z}_{n_2} \times \dots \times \mathbb{Z}_{n_r}$$

zugrunde. Dieser Sachverhalt soll im Folgenden erläutert werden.

Kartesisches Produkt von Ringen

Sind $R_1, R_2, \dots, R_r$ Ringe, so bildet das kartesische Produkt

$$R = R_1 \times R_2 \times \cdots \times R_r$$

mit den Verknüpfungen

$$(a_1, \dots, a_r) + (b_1, \dots, b_r) = (a_1 + b_1, \dots, a_r + b_r)$$

$$(a_1, \dots, a_r) \cdot (b_1, \dots, b_r) = (a_1 \cdot b_1, \dots, a_r \cdot b_r)$$

ebenfalls einen Ring.

Das Nullelement in R ist $(0, \dots, 0)$ und das Einselement ist $(1, \dots, 1)$.

Kommentar Gilt

$$R = R_1 \times R_2 \times \cdots \times R_r$$

so gilt für die Einheitengruppen (Menge der invertierbaren Elemente) der Ringe:

$$R^* = R_1^* \times R_2^* \times \cdots \times R_r^* \quad (1.7)$$

Ist $(a_1 + b_1, \dots, a_r + b_r) \in R^*$, so ist:

$$(a_1, \dots, a_r)^{-1} = (a_1^{-1}, \dots, a_r^{-1})$$

**Isomorphie von Ringen**

Eine bijektive Abbildung zwischen zwei Ringen R_1 und R_2

$$f : R_1 \rightarrow R_2$$

wird *Isomorphismus* genannt, falls für alle $a, b \in R_1$ gilt:

$$f(a + b) = f(a) + f(b)$$

$$f(a \cdot b) = f(a) \cdot f(b)$$

Gibt es einen Isomorphismus zwischen R_1 und R_2 , so heißen die Ringe *isomorph*. In Zeichen:

$$R_1 \cong R_2$$

Dem chinesischen Restsatz entspricht der folgende Satz.

Satz 1.10

Sind $n_1, n_2, \dots, n_r$ paarweise teilerfremde natürliche Zahlen, so gilt:

$$\mathbb{Z}_{n_1 n_2 \cdots n_r} \cong \mathbb{Z}_{n_1} \times \mathbb{Z}_{n_2} \times \cdots \times \mathbb{Z}_{n_r}$$

Beweis Die Abbildung

$$f: \mathbb{Z}_{n_1 n_2 \dots n_r} \rightarrow \mathbb{Z}_{n_1} \times \mathbb{Z}_{n_2} \times \dots \times \mathbb{Z}_{n_r}$$

mit

$$f(a) = (a \bmod n_1, \dots, a \bmod n_r)$$

ist ein Isomorphismus.

Überzeugen Sie sich bitte selbstständig von der Gültigkeit der Gleichungen

$$f(a+b) = f(a) + f(b)$$

$$f(a \cdot b) = f(a) \cdot f(b)$$

für alle $a, b \in \mathbb{Z}_{n_1 n_2 \dots n_r}$.

Da $\mathbb{Z}_{n_1 n_2 \dots n_r}$ und $\mathbb{Z}_{n_1} \times \mathbb{Z}_{n_2} \times \dots \times \mathbb{Z}_{n_r}$ die gleiche Anzahl $n = n_1 n_2 \dots n_r$ von Elementen besitzen, genügt es zum Beweis der Bijektivität die Injektivität zu überprüfen.

Zum Beweis der Injektivität seien $a, b \in \mathbb{Z}_{n_1 n_2 \dots n_r}$ mit $f(a) = f(b)$. Dann gilt für $i = 1, \dots, r$:

$$a \bmod n_i = b \bmod n_i \implies n_i \mid (a - b)$$

und wegen der Teilerfremdheit der n_i gilt auch:

$$n = n_1 n_2 \dots n_r \mid (a - b)$$

Wegen $|a - b| \leq n - 1$ folgt aus $n \mid (a - b)$ schließlich $a = b$. ■

Kommentar Beachten Sie bitte, dass die Surjektivität der im obigen Beweis definierten Funktion f auch mit dem chinesischen Restsatz (Satz 1.9) folgt.

Der Beweis des chinesischen Restsatzes beinhaltet darüber hinaus ein konstruktives Verfahren zur Berechnung von Urbildern für gegebene Elemente $(a_1, \dots, a_r)$ der Zielmenge von f . Die Elemente $e_1, \dots, e_r$ aus dem Beweis von Satz 1.9 entsprechen dabei den Urbildern von:

$$(1, 0, \dots, 0), (0, 1, 0, \dots, 0), \dots, (0, \dots, 0, 1)$$

◀

Wichtig Ein Isomorphismus bildet invertierbare Elemente auf invertierbare Elemente ab und wegen (1.7) gilt bezüglich der in Satz 1.10 angegebenen Isomorphie

$$\mathbb{Z}_{n_1 n_2 \dots n_r}^* \cong \mathbb{Z}_{n_1}^* \times \mathbb{Z}_{n_2}^* \times \dots \times \mathbb{Z}_{n_r}^* \quad (1.8)$$

(Die Isomorphie bezüglich der Einheitengruppen bezieht sich natürlich nur auf die multiplikative Verknüpfung.)

Insbesondere ergibt sich aus (1.8) die bereits im Studienheft Datensicherheit 01, Satz 2.13 behauptete Multiplikativität der Euler'schen ϕ -Funktion für teilerfremde Faktoren! ◀

Außerdem liefert (1.8) zusammen mit den Ergebnissen aus Abschnitt 1.2 eine strukturelle Beschreibung der Einheitengruppen $\mathbb{Z}_n^*$ für alle $n \in \mathbb{N}$:

Es sei $n \in \mathbb{N}$ mit $n > 1$ und

$$n = p_1^{e_1} \dots p_r^{e_r}$$

die Primfaktorzerlegung von n mit paarweise verschiedenen Primzahlen $p_1, \dots, p_r$ und positiven Exponenten $e_1, \dots, e_r \in \mathbb{N}$. Dann gilt bezüglich der multiplikativen Operation:

$$\mathbb{Z}_n^* \cong \mathbb{Z}_{p_1^{e_1}}^* \times \mathbb{Z}_{p_2^{e_2}}^* \times \dots \times \mathbb{Z}_{p_r^{e_r}}^* \quad (1.9)$$

Für ungerade Primzahlen p_i sind die Gruppen $\mathbb{Z}_{p_i^{e_i}}^*$ zyklisch. Für $p_i = 2$ und $e_i \geq 3$ ist die Struktur von $\mathbb{Z}_{p_i^{e_i}}^*$ in Satz 1.8 beschrieben.

Beispiel 1.3 $\mathbb{Z}_{143} \cong \mathbb{Z}_{11} \times \mathbb{Z}_{13}$

Für $n = 143 = 11 \cdot 13$ soll der im Beweis von Satz 1.10 beschriebene Isomorphismus

$$f: \mathbb{Z}_{143} \rightarrow \mathbb{Z}_{11} \times \mathbb{Z}_{13}$$

konkret nachvollzogen werden.

Zur Bestimmung der Urbilder $e_1, e_2 \in \mathbb{Z}_{143}$ von $(1, 0), (0, 1) \in \mathbb{Z}_{11} \times \mathbb{Z}_{13}$ bezüglich der Abbildung f kann der chinesische Restsatz zur Lösung von

$$e_1 \equiv 1 \pmod{11}$$

$$e_1 \equiv 0 \pmod{13}$$

bzw.

$$e_2 \equiv 0 \pmod{11}$$

$$e_2 \equiv 1 \pmod{13}$$

angewendet werden. Hierzu wird zunächst der erweiterte Euklid'sche Algorithmus auf die Zahlen $n_1 = m_2 = 11$ und $n_2 = m_1 = 13$ angewendet. Hierbei ergibt sich:

$$6 \cdot 11 + (-5) \cdot 13 = 1$$

Die gesuchten Lösungen der obigen Systeme von Kongruenzen sind daher:

$$e_1 = (-5 \cdot 13) \bmod 143 = 6 \cdot 13 = 78$$

$$e_2 = 6 \cdot 11 \bmod 143 = 66$$

Die Umkehrabbildung von f ist daher der Isomorphismus

$$f^{-1}: \mathbb{Z}_{11} \times \mathbb{Z}_{13} \rightarrow \mathbb{Z}_{143}, \quad f^{-1}((i, j)) = (78 \cdot i + 66 \cdot j) \bmod 143$$

und f^{-1} entspricht der folgenden Darstellung der Elemente aus $\mathbb{Z}_{143}$:

$$\mathbb{Z}_{143} = \{(78 \cdot i + 66 \cdot j) \bmod 143 \mid i \in \mathbb{Z}_{11}, j \in \mathbb{Z}_{13}\}$$

Schließlich soll auch noch die Einheitengruppe $\mathbb{Z}_{143}^*$ in $\mathbb{Z}_{143}$ mithilfe der Einheiten von $\mathbb{Z}_{11}$ und $\mathbb{Z}_{13}$ dargestellt werden.

Den primitiven Elementen $2 \in \mathbb{Z}_{11}$ und $2 \in \mathbb{Z}_{13}$ entsprechen in $\mathbb{Z}_{11}^* \times \mathbb{Z}_{13}^*$ die Elemente $(2, 1)$ und $(1, 2)$ und alle Elemente aus $\mathbb{Z}_{11}^* \times \mathbb{Z}_{13}^*$ lassen sich eindeutig in der Form

$$(2^i, 2^j) \quad \text{mit } i \in \mathbb{Z}_{10}, j \in \mathbb{Z}_{12} \quad (*)$$

darstellen.

Die Urbilder von $(2, 1)$ und $(1, 2)$ bezüglich f sind:

$$g_1 = (2 \cdot e_1 + 1 \cdot e_2) \bmod 143 = 79$$

$$g_2 = (1 \cdot e_1 + 2 \cdot e_2) \bmod 143 = 67$$

Die durch f induzierten Isomorphie (1.8)

$$\mathbb{Z}_{143}^* \cong \mathbb{Z}_{11}^* \times \mathbb{Z}_{13}^*$$

liefert eine (*) entsprechende Darstellung der Elemente von $\mathbb{Z}_{143}^*$:

$$\mathbb{Z}_{143}^* = \{(79^i \cdot 67^j) \bmod 143 \mid i \in \mathbb{Z}_{10}, j \in \mathbb{Z}_{12}\}$$

Für die (multiplikative) Ordnung eines Elements

$$(\alpha, \beta) \in \mathbb{Z}_{11}^* \times \mathbb{Z}_{13}^*$$

gilt:

$$o((\alpha, \beta)) = \text{kgV}(o(\alpha), o(\beta))$$

Ein Element maximaler Ordnung in $\mathbb{Z}_{11}^* \times \mathbb{Z}_{13}^*$ ist beispielsweise $(2, 2)$. Für das entsprechende Element in $\mathbb{Z}_{143}^*$

$$a = (2 \cdot e_1 + 2 \cdot e_2) \bmod 143 = 2$$

gilt $o(a) = \text{kgV}(10, 12) = 60$. Primitive Elemente besitzt $\mathbb{Z}_{143}^*$ nicht. ◀

Frage 11

Zeigen Sie: Sind $n_1, n_2 \in \mathbb{N}$ mit $n_1, n_2 \geq 2$ und $\text{ggT}(n_1, n_2) = 1$, so gilt für $n = n_1 \cdot n_2$:

$$\mu_n = \text{kgV}(\mu_{n_1}, \mu_{n_2})$$

Frage 12

Bestimmen Sie die $n \in \mathbb{N}, n \geq 2$, für die $\mathbb{Z}_n^*$ zyklisch ist, d. h. für die ein $g \in \mathbb{Z}_n$ mit $o(g) = |\mathbb{Z}_n^*|$ existiert.

Frage 13

Es sei f der Ringisomorphismus:

$$f: \mathbb{Z}_{1232} \rightarrow \mathbb{Z}_{16} \times \mathbb{Z}_7 \times \mathbb{Z}_{11}, \quad f(x) = (x \bmod 16, x \bmod 7, x \bmod 11)$$

- (1) Bestimmen Sie die Abbildungsvorschrift der Umkehrfunktion von f . Finden Sie hierzu $\alpha, \beta, \gamma \in \mathbb{Z}_{1232}$ mit:

$$f^{-1}(i, j, k) = (\alpha \cdot i + \beta \cdot j + \gamma \cdot k) \bmod 1232 \quad \text{for all } (i, j, k) \in \mathbb{Z}_{16} \times \mathbb{Z}_7 \times \mathbb{Z}_{11}$$

- (2) Bestimmen Sie $\mu_{1232} = \max\{o(a) \mid a \in \mathbb{Z}_{1232}^*\}$.
- (3) Geben Sie ein Element $g_m \in \mathbb{Z}_{1232}^*$ mit $o(g_m) = \mu_{1232}$ an.

1.4 Mathematischer Exkurs: Elemente ungerader und gerader Ordnung in $\mathbb{Z}_n$

Ist für eine natürliche Zahl $n \in \mathbb{N}$ mit $n > 1$ die Primfaktorzerlegung

$$n = p_1^{e_1} \cdots p_r^{e_r}$$

mit paarweise verschiedenen Primzahlen $p_1, \dots, p_r$ und positiven Exponenten $e_1, \dots, e_r \in \mathbb{N}$ bekannt so kann die Einheitengruppe $\mathbb{Z}_n^*$ mithilfe der durch (1.8) gegebenen Isomorphie

$$\mathbb{Z}_n^* \cong \mathbb{Z}_{p_1^{e_1}}^* \times \mathbb{Z}_{p_2^{e_2}}^* \times \dots \times \mathbb{Z}_{p_r^{e_r}}^*$$

und unter Nutzung der Ergebnisse aus Abschnitt 1.2 untersucht werden.

Im Folgenden beschränken wir uns auf den Spezialfall, dass

$$n = p \cdot q$$

das Produkt von zwei verschiedenen ungeraden Primzahlen p und q ist. Im Zusammenhang mit dem RSA-Verfahren ist es interessant, eine Abschätzung der Anzahl von Elementen $a \in \mathbb{Z}_n^*$ zu gewinnen, die eine gerade Ordnung besitzen, für die aber

$$-1 \notin \langle a \rangle \quad (1.10)$$

gilt.

Beginnen wir mit einigen Beispielen, die die Untersuchung von $\mathbb{Z}_{143}^*$ aus Beispiel 1.3 fortsetzen.

Beispiel 1.4 Elemente ungerader Ordnung in $\mathbb{Z}_{143}^*$

Mit dem Isomorphismus f aus Beispiel 1.3 können die Elemente ungerader Ordnung in $\mathbb{Z}_{143}^*$ als Urbilder der Elemente ungerader Ordnung in $\mathbb{Z}_{11}^* \times \mathbb{Z}_{13}^*$ beschrieben werden.

Die Elemente ungerader Ordnung in $\mathbb{Z}_{11}^* \times \mathbb{Z}_{13}^*$ sind:

$$(2^{2i}, 2^{4j}) = (4^i, 3^j) \quad \text{mit } i \in \mathbb{Z}_5, j \in \mathbb{Z}_3$$

In $\mathbb{Z}_{143}^*$ gibt es daher auch genau 15 Elemente ungerader Ordnung, die wie folgt aufgelistet werden können:

$$79^{2i} \cdot 67^{4j} \bmod 143 = 92^i \cdot 133^j \bmod 143 \quad \text{mit } i \in \mathbb{Z}_5, j \in \mathbb{Z}_3$$

Beispiel 1.5 Elemente $a \in \mathbb{Z}_{143}^*$ mit $o(a) = 2$

Die Elemente der Ordnung 2 in $\mathbb{Z}_{11}^* \times \mathbb{Z}_{13}^*$ sind:

$$(-1, -1) = (10, 12), \quad (-1, 1) = (10, 1) \quad \text{und} \quad (1, -1) = (1, 12)$$

Mit den Bezeichnungen aus Beispiel 1.3 sind die entsprechenden Elemente der Ordnung 2 in $\mathbb{Z}_{143}^*$:

$$-1 \bmod 143 = 142$$

$$(-e_1 + e_2) \bmod 143 = 131$$

$$(e_1 - e_2) \bmod 143 = 12$$

Beachten Sie, dass die Differenz oder Summe von 131 oder 12 mit 1 immer eine Zahl liefert, die entweder von 11 oder von 13 geteilt wird. Das entspricht der Tatsache, dass

$$\pm(-1, 1) \pm (1, 1)$$

immer in einer Komponente 0 ist. (Die andere Komponente ist ± 2 .)

Die Beobachtung am Ende von Beispiel 1.5 lässt sich allgemein dazu verwenden, die Primfaktoren einer aus zwei verschiedenen Primzahlen p und q zusammengesetzten Zahl $n = p \cdot q$ zu ermitteln, falls ein Element $a \in \mathbb{Z}_n^*$ mit $a \neq -1$ und $o(a) = 2$ gegeben ist.

Satz 1.11

Es sei $n = p \cdot q$ mit zwei verschiedenen Primzahlen p und q . Weiterhin sei $a \in \mathbb{Z}_n^*$ mit $a \neq -1$ und $o(a) = 2$. Dann gilt:

$$\{p, q\} = \{\text{ggT}(n, a-1), \text{ggT}(n, a+1)\}$$

Beispiel 1.6 Elemente $a \in \mathbb{Z}_{143}^*$ gerader Ordnung mit $a^{\frac{o(a)}{2}} \neq -1$

Mit den Bezeichnungen aus Beispiel 1.3 entsprechen den Elementen gerader Ordnung in $\mathbb{Z}_{143}^*$ die folgenden Elemente in $\mathbb{Z}_{11}^* \times \mathbb{Z}_{13}^*$:

- (i) $(2^i, 2^j)$ mit beliebigen $i \in \mathbb{Z}_{10}$ und ungeraden $j \in \mathbb{Z}_{12}$
- (ii) $(2^i, 2^j)$ mit geraden $i \in \mathbb{Z}_{10}$ und $j \in \{2, 6, 10\}$
- (iii) $(2^i, 2^j)$ mit ungeraden $i \in \mathbb{Z}_{10}$ und $j \in \{2, 6, 10\}$
- (iv) $(2^i, 2^j)$ mit ungeraden $i \in \mathbb{Z}_{10}$ und $j \in \{0, 4, 8\}$

Ist $(\alpha, \beta) = (2^i, 2^j)$ wie in (i), so ist die Ordnung von (α, β) durch 4 teilbar und es gilt:

$$(\alpha, \beta)^{\frac{o((\alpha, \beta))}{2}} = (\alpha, \beta)^{15 \cdot 2} = (1, -1)$$

Ist $(\alpha, \beta) = (2^i, 2^j)$ wie in (ii), so gilt:

$$(\alpha, \beta)^{\frac{o((\alpha, \beta))}{2}} = (\alpha, \beta)^{15} = (1, -1)$$

Ist $(\alpha, \beta) = (2^i, 2^j)$ wie in (iii), so gilt:

$$(\alpha, \beta)^{\frac{o((\alpha, \beta))}{2}} = (\alpha, \beta)^{15} = (-1, -1)$$

Ist $(\alpha, \beta) = (2^i, 2^j)$ schließlich wie in (iv), so gilt:

$$(\alpha, \beta)^{\frac{o((\alpha, \beta))}{2}} = (\alpha, \beta)^{15} = (-1, 1)$$

Von den insgesamt 105 Elementen gerader Ordnung entsprechen den 15 Elementen aus (iii) den Elementen gerader Ordnung $a \in \mathbb{Z}_{143}^*$ mit $a^{\frac{o(a)}{2}} = -1$.

Die insgesamt 90 Elemente aus (i), (ii) und (iv) entsprechen den Elementen gerader Ordnung $a \in \mathbb{Z}_{143}^*$ mit $a^{\frac{o(a)}{2}} \neq -1$. 

Satz 1.12

Es sei $n = p \cdot q$ mit zwei verschiedenen Primzahlen p und q . Dann gilt für mindestens die Hälfte aller Elemente $a \in \mathbb{Z}_n^*$:

$$2 \mid o(a) \quad \text{und} \quad a^{\frac{o(a)}{2}} \neq -1 \quad (1.11)$$

Beweis Es sei

$$\mathbb{Z}_n^* \cong \mathbb{Z}_p^* \times \mathbb{Z}_q^* = \langle \gamma_1 \rangle \times \langle \gamma_2 \rangle$$

mit primitiven Elementen $\gamma_1 \in \mathbb{Z}_p^*$ und $\gamma_2 \in \mathbb{Z}_q^*$.

Weiter sei

$$o(\gamma_1) = p-1 = u_1 \cdot 2^{i_1}$$

$$o(\gamma_2) = q-1 = u_2 \cdot 2^{i_2}$$

mit ungeraden Zahlen $u_1, u_2 \in \mathbb{N}$.

Ist $i_1 < i_2$, so erfüllen unter anderem die folgenden Elemente die Bedingung (1.11):

$$(\gamma_1^i, \gamma_2^j) \quad \text{mit beliebigen } i \in \mathbb{Z}_{p-1} \text{ und ungeraden } j \in \mathbb{Z}_{q-1}$$

Die Anzahl dieser Elemente ist $\frac{|\mathbb{Z}_n^*|}{2}$. In Beispiel 1.6 sind dies die Elemente in (i) und wie in Beispiel 1.6 gibt es für $i_1 < i_2$ noch weitere Elemente, die die Bedingung (1.11) erfüllen.

Ist $i_1 = i_2$, so erfüllen zumindest die Elemente

$$(\gamma_1^i, \gamma_2^j) \quad \text{mit ungeraden } i \in \mathbb{Z}_{p-1} \text{ und geraden } j \in \mathbb{Z}_{q-1}$$

sowie

$$(\gamma_1^i, \gamma_2^j) \quad \text{mit geraden } i \in \mathbb{Z}_{p-1} \text{ und ungeraden } j \in \mathbb{Z}_{q-1}$$

die Bedingung (1.11). Die Anzahl dieser Elemente beträgt insgesamt $\frac{|\mathbb{Z}_n^*|}{2}$.

Ist $i_1 > i_2$, so folgt die Behauptung genauso wie in dem Fall, dass $i_1 > i_2$ gilt. ■

Der folgende Satz ist im Zusammenhang mit dem RSA-Verfahren von Interesse. Ausgehend von einer Zahl $n = p \cdot q$ mit großen und zunächst nicht bekannten Primfaktoren p und q zeigt er, dass bei Kenntnis eines beliebigen Vielfachen v von

$$\text{kgV}(p-1, q-1)$$

eine Zahl $a \in \mathbb{Z}_n^*$ mit gerader Ordnung leicht (effizient) gefunden werden kann, für die (1.10) gilt. Mit dieser Zahl können dann mit einer Anwendung von Satz 1.11 auch die Primfaktoren p und q leicht berechnet werden.

Satz 1.13 (Bestimmung der Primfaktoren p, q von $n = p \cdot q$ bei Kenntnis eines Vielfachen von $\text{kgV}(p-1, q-1)$)

Gegeben ist $n = p \cdot q$ mit unbekannten verschiedenen Primzahlen p und q . Weiterhin ist ein $v \in \mathbb{N}$ mit

$$\text{kgV}(p-1, q-1) \mid v \quad (1.12)$$

gegeben.

Dann lassen sich die Primfaktoren p und q mit folgendem Algorithmus effizient bestimmen:

- (1) Berechne den größten ungeraden Teiler u von v :
Dividiere hierzu v wiederholt durch 2 bis die Zerlegung

$$v = u \cdot 2^i \quad \text{mit } 2 \nmid u \quad (1.13)$$

gefunden ist.

Beachte: Wegen (1.12) gilt $o(a) \mid v$ für alle $a \in \mathbb{Z}_n^*$ und daher mit (1.13):

$$a^{u \cdot 2^i} = 1 \quad (1.14)$$

- (2) Wähle zufällig ein $a \in \mathbb{Z}_n \setminus \{0\}$.
(3) Berechne $g = \text{ggT}(n, a)$. Gilt $g \neq 1$, so ist $g = p$ oder $g = q$ und die Aufgabe der Primfaktorzerlegung ist gelöst. Der Algorithmus wird beendet.
(4) Berechne a^u . Ist $a^u = 1$, so gehe zu (2).
(5) Bestimme $a^{u \cdot 2^j}$ mit $j \in \mathbb{N}_0$ und:

$$a^{u \cdot 2^j} \neq 1 \quad \text{und} \quad a^{u \cdot 2^{j+1}} = 1 \quad (1.15)$$

Beachte: Wegen (1.14) und $a^u \neq 1$ (vgl. mit Schritt (4)) gibt es ein eindeutig bestimmtes $j \in \mathbb{N}_0$ mit $j < i$ welches (1.15) erfüllt.

Ausgehend von der in (4) berechneten Zahl a^u lässt sich $a^{u \cdot 2^j}$ durch fortgesetzte Quadratur bestimmen:

$$a^u, \quad (a^u)^2 = a^{u \cdot 2}, \quad (a^{u \cdot 2})^2 = a^{u \cdot 2^2}, \dots, \quad (a^{u \cdot 2^{j-1}})^2 = a^{u \cdot 2^j}, \quad (a^{u \cdot 2^j})^2 = 1$$

- (6) Ist $a^{u \cdot 2^j} = -1$, so gehe zu (2).
(7) Die Primfaktoren p und q von n sind:

$$\text{ggT}(n, a^{u \cdot 2^j} + 1) \quad \text{und} \quad \text{ggT}(n, a^{u \cdot 2^j} - 1)$$

Beweis Die Schritte (2)–(6) bilden eine Schleife, die in (6) genau dann verlassen wird, falls die in (2) zufällig gewählte Zahl a folgende Eigenschaften besitzt:

- $a \in \mathbb{Z}_n^*$ (Sonst wäre der Algorithmus bereits in (3) erfolgreich beendet worden.)
- $2 \mid o(a)$ (Sonst hätte bereits in (4) ein Rücksprung zu (2) stattgefunden.)
- Für die eindeutig bestimmte Involution in $\langle a \rangle$ gilt:

$$a^{\frac{o(a)}{2}} = a^{u \cdot 2^j} \neq -1$$

Unter den obigen Voraussetzungen an a folgt mit Satz 1.11, dass mit der Involution $a^{u \cdot 2^j}$ die Bestimmung der Primfaktoren p und q mit den Berechnungen der größten gemeinsamen Teiler in (7) möglich ist.

Natürlich liefert der Algorithmus nur dann ein effizientes Verfahren zur Bestimmung von p und q , falls zu erwarten ist, dass nur wenige verschieden zufällige Werte für a in (2) ausgewählt werden müssen, bis ein a die oben genannten Bedingungen erfüllt. Tatsächlich erfüllen entsprechend Satz 1.12 mindestens die Hälfte der Elemente aus $\mathbb{Z}_n^*$ die obigen Bedingungen, so dass durchschnittlich weniger als zwei Durchläufe von (2) zu erwarten sind! ■

Das Diffie-Hellman-Schlüsselaustauschverfahren

2

2.1	Exkurs: Berechnung modularer Potenzen	30
2.2	Exkurs: Angriffe auf das DH-Verfahren	34

Um Daten mithilfe eines symmetrischen Verschlüsselungsverfahrens vertraulich übertragen zu können, müssen Alice und Bob zunächst einen Schlüssel k_{AB} austauschen, der zur Ver- und Entschlüsselung der Nachrichten genutzt werden soll. Gleiches gilt für den Fall, dass die Nachrichtenauthentizität mithilfe eines MAC-Verfahrens gewährleistet werden soll.

Sicherlich ist es keine gute Idee, den Schlüssel k_{AB} über den gleichen (potentiell unsicheren) Kanal zu übertragen, über den anschließend die verschlüsselten Nachrichten übertragen werden: Kann Charlie den Kanal abhören, so könnte er ja den Schlüssel ebenfalls mitlesen und die anschließend verschlüsselt übertragenen Nachrichten auch entschlüsseln.

Vor einer symmetrischen Verschlüsselung oder MAC-Berechnung könnten Alice und Bob einen Schlüssel über einen anderen sicheren Kanal (Post, persönliche Übergabe eines Speichermediums mit dem Schlüssel, etc.) austauschen. Dies ist nicht komfortabel und erlaubt keinen Aufbau eines sicheren Kanals in Echtzeit, wie dies beispielsweise beim Aufruf einer Webseite über das HTTPS-Protokoll notwendig ist.

Eine bahnbrechende Idee zur Lösung des Schlüsselaustauschproblems wurde 1976 von W. Diffie und M. E. Hellman in ihrer Arbeit *New Directions in Cryptography* vorgestellt. Dieses Diffie-Hellman-Verfahren (kurz DH-Verfahren) ist eine konkrete Realisierung des Konzepts eines *Public Key Distribution Systems*, welches zuvor bereits von R. C. Merkle entwickelt wurde. Die Arbeit von Diffie und Hellman stellte einen wesentlichen Beitrag zur Begründung eines neuen Teilgebiets in der Kryptographie dar, welches als *asymmetrische Kryptographie* (*Public Key Cryptography*) bezeichnet wird. (In der Einführung zu der im Jahr 2002 erneut veröffentlichten Arbeit *An Overview of Public Key Cryptography* („Landmark Article“ im *IEEE Communications Magazine, 50th Anniversary Commemorative Issue, May 2002*), die M. E. Hellman ursprünglich 1978 publizierte, schlägt Hellman deshalb vor, das DH-Verfahren als Diffie-Hellman-Merkle-Verfahren (DHM-Verfahren) zu bezeichnen. Da sich dieser Vorschlag jedoch nicht allgemein durchgesetzt hat, wird im Folgenden die Bezeichnung *DH-Verfahren* verwendet.)

Kennzeichnend für Verfahren der asymmetrischen Kryptographie ist die Verwendung von *Schlüsselpaaren*, die jeweils aus einem privaten und einem öffentlichen Schlüssel (*private key* k_{priv} und *public key* k_{pub}) bestehen:

$$k = (k_{priv}, k_{pub})$$

Für Schlüsselpaare, die in asymmetrischen kryptographischen Verfahren zum Einsatz kommen, sollten folgende grundlegende Eigenschaften gelten:

- Ein Schlüsselpaar ist genau einem/einer „Besitzer/in“ zugeordnet. Alice besitzt beispielsweise das Schlüsselpaar:

$$k_A = (k_{A,priv}, k_{A,pub})$$

- Niemals hat jemand anderes als Alice Zugriff auf $k_{A,priv}$.
- Konsequenterweise muss Alice $k_{A,priv}$ selbständig generieren.
- Unter Verwendung von $k_{A,priv}$ berechnet Alice auch $k_{A,pub}$ mithilfe eines effizient ausführbaren Algorithmus.
- Es gibt keinen effizient ausführbaren Algorithmus mit dem $k_{A,priv}$ bestimmt werden kann, falls nur $k_{A,pub}$ bekannt ist.

Bei dem DH-Verfahren besitzen Alice und Bob zunächst jeweils ein Schlüsselpaar, mit denen eine *Schlüsselvereinbarung* für einen symmetrischen Schlüssel (auch Schlüsselaustausch genannt) nach Austausch der öffentlichen Schlüssel möglich ist:

Diffie-Hellman-Verfahren (DH-Verfahren)

- Alice und Bob vereinbaren zunächst zwei Parameter, eine große natürliche Zahl n und eine Zahl:

$$g \in \mathbb{Z}_n^*$$

Die Ordnung $o(g)$ von g muss hierbei mindestens durch eine sehr große Primzahl geteilt werden.

Alle weiteren Rechnungen werden in $\mathbb{Z}_n$ ausgeführt.

(Häufig nutzen Alice und Bob DH-Parameter n und g , die in einem Standard speziell zur Nutzung mit dem DH-Verfahren spezifiziert sind.)

- Alice bestimmt eine zufällige Zahl $e_A \in \mathbb{Z}_n^*$ als privaten Schlüssel, d. h.:

$$k_{A,priv} = e_A$$

und berechnet dann den öffentlichen Schlüssel in $\mathbb{Z}_n$:

$$k_{A,pub} = g^{e_A} \in \mathbb{Z}_n$$

Das Schlüsselpaar von Alice ist somit:

$$(k_{A,priv}, k_{A,pub}) = (e_A, g^{e_A})$$

- Bob bestimmt ebenso eine zufällige Zahl $e_B \in \mathbb{Z}_n^*$ und damit das Schlüsselpaar:

$$(k_{B,priv}, k_{B,pub}) = (e_B, g^{e_B})$$

- Alice und Bob tauschen ihre öffentlichen Schlüssel aus, d. h. Alice übermittelt g^{e_A} an Bob und Bob sendet g^{e_B} an Alice.

- Alice berechnet in $\mathbb{Z}_n$:

$$s_A := (g^{e_B})^{e_A}$$

Bob berechnet in $\mathbb{Z}_n$:

$$s_B := (g^{e_A})^{e_B}$$

- Es gilt

$$s_A = (g^{e_B})^{e_A} = g^{e_B \cdot e_A} = g^{e_A \cdot e_B} = (g^{e_A})^{e_B} = s_B$$

und $s_A = s_B$ ist der zwischen Alice und Bob vereinbarte Schlüssel, beziehungsweise ein sogenanntes *shared secret* aus dem weitere Schlüssel zur Verwendung in symmetrischen kryptographischen Verfahren geeignet extrahiert beziehungsweise abgeleitet werden.

Der „Witz“ des DH-Verfahrens besteht darin, dass Alice und Bob ihre öffentlichen Schlüssel über den gleichen Kanal austauschen können, über den sie anschließend ihre mit $s_A = s_B$ verschlüsselten Nachrichten austauschen wollen. Grundlage hierfür ist das sogenannte *Diskrete-Logarithmus-Problem (DL-Problem)*:

Diskrete-Logarithmus-Problem (DL-Problem)

Gegeben seien $n \in \mathbb{N}$, $g \in \mathbb{Z}_n^*$ und eine Potenz von g in $\mathbb{Z}_n$:

$$a \in \langle g \rangle := \{g^e \mid e \in \mathbb{N}\}$$

Die Aufgabenstellung, unter diesen Voraussetzungen einen geeigneten Exponenten e_A mit

$$a = g^{e_A}$$

zu finden, wird als DL-Problem bezeichnet.

Es wird vermutet, dass bei einer geeigneten Wahl von n und g kein Algorithmus existiert, der das DL-Problem effizient löst.

Nehmen wir nun an, dass Alice und Bob die Parameter n und g so gewählt haben, dass das DL-Problem nicht effizient gelöst werden kann. Falls Charlie den von Alice und Bob genutzten Kanal abhören kann, so kann Charlie zwar die öffentlichen Schlüssel g^{e_A} und g^{e_B} mitlesen, wird jedoch wegen des DL-Problems die Exponenten e_A und e_B nicht bestimmen können. (Könnte Charlie dies tun, könnte Charlie natürlich auch das shared secret $s_A = s_B$ berechnen und das DH-Verfahren wäre „witzlos“.)

Kann Charlie aber vielleicht allein mithilfe von g^{e_A} und g^{e_B} den Wert des shared secrets $s_A = s_B = g^{e_A \cdot e_B}$ effizient berechnen? Leider ist eine definitive Antwort auf diese als Diffie-Hellman-Problem bezeichnete Fragestellung ebenso wenig bekannt, wie für das DL-Problem. Wie oben besprochen, würde eine Lösung des DL-Problems eine Lösung des DH-Problems liefern. Andererseits konnte bisher nicht vollständig geklärt werden, ob die Bestimmung einer Lösung des DH-Problems tatsächlich die gleiche Komplexität aufweist, wie die Bestimmung einer Lösung des DL-Problems. Im Allgemeinen wird dies vermutet:

Diffie-Hellman-Problem (DH-Problem)

Gegeben seien $n \in \mathbb{N}$, $g \in \mathbb{Z}_n^*$ und zwei Potenzen von g in $\mathbb{Z}_n$:

$$g^{e_A}, g^{e_B} \in \langle g \rangle$$

Die Aufgabenstellung, unter diesen Voraussetzungen die Zahl

$$g^{e_A \cdot e_B}$$

zu finden, wird als DH-Problem bezeichnet.

Es wird vermutet, dass bei einer Wahl von n und g , für die das DL-Problem nicht effizient lösbar ist, auch kein Algorithmus existiert, der das DH-Problem effizient löst.

Unter Voraussetzung der Gültigkeit des DH-Problems kann Charlie also trotz Kenntnis der beiden öffentlichen Schlüssel das shared secret $s_A = s_B$ nicht bestimmen.

Nachdem Sie nun ein grundsätzliches Verständnis über die Funktionsweise des DH-Verfahrens besitzen, sollen die einzelnen Schritte nochmals etwas detaillierter behandelt werden. Beginnen wir mit der Auswahl geeigneter DH-Parameter:

Beispiel 2.1 (DH-Parameter)

DH-Parameter sind beispielsweise im RFC 7919 (Negotiated Finite Field Diffie-Hellman Ephemeral Parameters for Transport Layer Security (TLS)) im Appendix A zu finden. Die dort angegebenen fünf verschiedenen Zahlen für n sind Primzahlen mit Bitlängen von 2048, 3072, 4096, 6144 und 8192. Die fünf Zahlen sind sogar sogenannte *sichere Primzahlen*, bei denen auch $\frac{p-1}{2}$ eine Primzahl ist. Solche Primzahlen werden auch als *Sophie-Germain-Primzahlen* bezeichnet.

In allen fünf Fällen ist $g = 2$ und es gilt $o(g) = \frac{p-1}{2}$. Zum Beispiel ist der spezifizierte Wert für n mit einer Länge von 2048 Bitstellen:

```
n2048 = 32317006071311007300153513477825163362488057133489
        07517458843413926980683413621000279205636264016468
        54585563579353308169288290230805734726252735547424
        61245741026202527916572972862706300325263428213145
        76693141422365422094111134862999165747826803423055
        30863490506355577122191878903327295696961297438562
        41741236237225197346402691855797767976823014625397
        93305801522685873076119753243646747585546071504389
        68449403661304976978128542959586595975670512838521
        32784468522925504568272879113720098931873959143374
        17583782600027803497319855206060753323412260325468
        40881200311059074842810039949669561196969562486290
        32338072839127039
```

Zu beachten ist, dass der Aufwand eines Brute-Force-Angriffs auf das DL-Problem nicht direkt mit der Bitlänge von n abzuschätzen ist. In RFC 7919 wird beispielsweise die Größe des effektiv zu durchsuchenden Raumes als „*estimated symmetric-equivalent strength*“ bezeichnet und für $n = n_{2048}$ mit 2^{103} abgeschätzt. ◀

Kommentar Vielleicht haben Sie sich gefragt, wie es möglich sein kann, für eine so große Zahl wie $n = n_{2048}$ aus Beispiel 2.1 zu überprüfen, dass es sich tatsächlich um eine Primzahl handelt?

Ein Algorithmus, der für eine gegebene natürliche Zahl n entscheiden kann, ob n eine Primzahl ist, wird *Primzahltest* genannt. Als Primzahltest könnten beispielsweise die ungeraden ganzen Zahlen $\leq \sqrt{n}$ durchlaufen und daraufhin überprüft werden, ob sie n ohne Rest teilen. Ein solcher Test würde allerdings bei einer Anwendung auf $n = n_{2048}$ selbst bei einem Einsatz aller auch nur theoretisch vorstellbar zur Verfügung stehenden Rechnerressourcen eine Laufzeit besitzen, die astronomische Größenordnungen wie das Alter unseres Universums bei weitem überträfe.

Die Frage, ob es überhaupt einen Primzahltest gibt, dessen Laufzeit polynomial durch die Stellenzahl (Bitlänge) der zu überprüfenden Zahl beschränkt ist, war lange Zeit offen. Im Jahr 2002 konnte sie von M. Agrawal, N. Kayal und N. Saxena in ihrer Arbeit *PRIMES is in P* positiv entschieden werden. Der entsprechende Primzahltest heißt *AKS-Test*.

Trotz polynomialer Laufzeit, dauert der AKS-Test für Primzahlen, die in der asymmetrischen Kryptographie benötigt werden, viel zu lange. (Auch ein Polynom kann sehr große Werte annehmen.)

In der Praxis kommen daher sogenannte *probabilistische Primzahltests*, die mit einer Fehlerwahrscheinlichkeit p parametrisiert werden können, zum Einsatz. Für jede natürliche Zahl n liefert ein probabilistischer Primzahltest eine der beiden Aussagen zurück:

- n ist keine Primzahl.

Dann ist n definitiv keine Primzahl. Der Algorithmus liefert aber in der Regel keinen echten Teiler von n .

■ n ist wahrscheinlich eine Primzahl.

Bei hinreichend kleinem p kann dann davon ausgegangen werden, dass n eine Primzahl ist, da unter allen Zahlen, für die der Algorithmus mit dieser Aussage endet, der Anteil von Zahlen, die keine Primzahl sind, kleiner als p ist.



Frage 14

Der DH-Parameter $p = n_{2048}$ aus Beispiel 2.1 ist in RFC 7919 folgendermaßen definiert:

$$p = 2^{2048} - 2^{1984} + \left(\lfloor 2^{1918} e \rfloor + 560316 \right) \cdot 2^{64} - 1$$

Benutzen Sie die Java-Klasse `BigInteger` zur Bearbeitung folgender Aufgaben, wobei eine Zahl als Primzahl „verifiziert“ gilt, falls der Methodenaufruf `isProbablePrime(50)` für eine `BigInteger`-Repräsentation der Zahl den Rückgabewert `true` liefert.

- (i) Verifizieren Sie, dass p eine Primzahl ist.
- (ii) Verifizieren Sie, dass $q = \frac{p-1}{2}$ eine Primzahl ist.
- (iii) Bestimmen Sie die Ordnung von $g = 2$ in $\mathbb{Z}_p^*$.
- (iv) Bestimmen Sie das kleinste Element $a \in \mathbb{Z}_p^*$ mit $o(a) = p - 1$.

Frage 15

Bezüglich der DH-Parameter $(n, g) = (n_{2048}, 2)$ aus Beispiel 2.1 besitzt Alice den privaten Schlüssel

```
720B062C8AFACCF8FBBA2569E3E427EFEA2E53E2323B7B2B715510016CDD5B3C
26996C16C2348054C010830BBA9E388B7ADF84BCAAF34611DD31DE983A7DDB16
537F4ED51BA9546904A348E4601071EB2DF69565AA96CA00133DC744316032AB
066B24F6BC752C746E0FF2183C559F668DE0E804C8482ECEFAFC3F3A328D8F19
51487516C48C8A96227C825BE4B8A266AFDB16B0ABE01140D639260758612CE0
231A7FDA97DD66E1762E7F4A911B606DF06B006DCDE827CF64C00210043F8D4E
62A6551983C3A609DFC2302A6C910E9AE502949A29979E6351D56D3C326D0653
3AABB34B1D9B84EB254F3AC8F644FE4F0518ABE21BBA77125F95A8C991204B3A
```

und Bob den öffentlichen Schlüssel:

```
3661E742E231DA656AEAF29512DDDC333CA7733B1529330FD9167C094C73A3A0
9CD1A480198A02BF028A4EBF37AAC485ACB51A10DB8D30545A2EB8211459F5F1
4E028A7626517ACB85172B9EDCE1E353ACE7CB0A6F5696988CC034AAFC35B92E
D2C2E54E804CA03F79744E6F07CD1C392D9B6DB56F09826B92738095999360FF
8BBC3775579034620E00E0D2BC96E0C56EF2FB1A56D8C88F4F0D124B2E745930
33DCDD00DF146D3A0BD0FFD81C29CC265FE3C84FC2A3E852D8788FF97634F0E0
67E1A16AA524F9C7231AC9CEDB9C1B06BB20FA279809C357FDEDF15A7A006E35
8F1F4DD6132392B9449D1C83277BBD20261A59D5F81B4F3B0DA0FDD47A579417
```

Berechnen Sie das gemeinsame Geheimnis von Alice und Bob.

Frage 16

Bob hat das Prinzip der DH-Schlüsselerzeugung nicht verstanden. Er hat für seinen privaten Schlüsselwert e_B , zu dem der öffentlichen Schlüssel aus Frage 15 korrespondiert, einen `int`-Wert ($-2^{31} \leq e_B < 2^{31}$) gewählt. Bestimmen Sie Bobs privaten Schlüssel.

Beispiel 2.2

Realistische Beispiele mit Zahlen, die mehrere hundert Dezimalstellen besitzen, können natürlich nur mit einem Computer durchgerechnet werden. Prinzipiell lassen sich die Rechnungen jedoch mit kleinen Zahlen nachvollziehen:

Alice und Bob wollen einen Schlüssel (shared secret) mithilfe des DH-Verfahrens vereinbaren. Hierzu verabreden sie, die „DH-Parameter“

$$(n, g) = (47, 2)$$

zu verwenden. (Beachten Sie, dass n eine sichere Primzahl ist.)

Alice benutzt einen Zufallszahlengenerator um ihren privaten Schlüsselwert zu erzeugen. Dieser sei beispielsweise:

$$k_{A,priv} = e_A = 17$$

Alice berechnet nun ihren zu $k_{A,priv}$ gehörenden öffentlichen Schlüssel:

$$k_{A,pub} = g^{e_A} \bmod n = 2^{17} \bmod 47 = 131072 \bmod 47 = 36$$

Alice besitzt somit das Schlüsselpaar:

$$(k_{A,priv}, k_{A,pub}) = (17, 36)$$

Ebenso erzeugt sich Bob ein Schlüsselpaar. Dieses sei:

$$(k_{B,priv}, k_{B,pub}) = (13, 14)$$

(Überprüfen Sie, dass dieses Zahlenpaar tatsächlich ein Schlüsselpaar ist!)

Nachdem Alice und Bob ihre öffentlichen Schlüssel ausgetauscht haben, können sie ihren gemeinsamen Schlüssel $k_{A,B}$ berechnen. Alice rechnet hierzu

$$\begin{aligned} k_{A,B} &= k_{B,pub}^{e_A} \bmod n = 14^{17} \bmod 47 \\ &= 30491346729331195904 \bmod 47 = 28 \end{aligned}$$

und Bob:

$$\begin{aligned} k_{A,B} &= k_{A,pub}^{e_B} \bmod n = 36^{13} \bmod 47 \\ &= 170581728179578208256 \bmod 47 = 28 \end{aligned}$$

Bei der Berechnung der Potenzen in Beispiel 2.2 fällt auf, dass diese bereits recht groß werden und mit einem Taschenrechner nicht mehr in jedem Fall exakt berechnet werden können. Das exakte Ergebnis wird aber benötigt, um den Rest nach Multiplikation mit n bestimmen zu können.

Bei den in den Anwendungen vorkommenden Zahlen haben die Exponenten e eine Länge von mehreren tausend Bitstellen, d. h. $e > 2^{1000}$. Für eine Potenz mit dem Exponenten e ergibt sich dann folgende Abschätzung der Bitlänge:

$$\log_2(a^e) = e \cdot \log_2(a) \geq e > 2^{1000} \approx 10^{300}$$

Die Anzahl der Bitstellen von a^e sprengt damit selbst astronomische Vorstellungen. (Zum Vergleich wird die Anzahl der Atome im sichtbaren Universum mit 10^{80} abgeschätzt.)

Eine Berechnung von Potenzen mit den beim DH-Verfahren vorkommenden Exponenten ist also in den ganzen Zahlen unmöglich. Dies ist aber auch nicht nötig, da das Ergebnis nicht als ganze Zahl sondern als Zahl in $\mathbb{Z}_n$ benötigt wird. In $\mathbb{Z}_n$ können Potenzen effektiv berechnet werden. Hierbei treten als Zwischenergebnisse immer nur Zahlen auf, die höchstens doppelt so viele Stellen wie n besitzen und dann sofort modulo n reduziert werden.

Beispiel 2.3

Soll beispielsweise (wie in Beispiel 2.2) $36^{13} \bmod 47$ berechnet werden, so können zunächst ausgehend von $a = 36$ die folgenden Quadrate berechnet werden:

$$36^2 \bmod 47 = 1296 \bmod 47 = 27$$

$$36^4 \bmod 47 = 27^2 \bmod 47 = 729 \bmod 47 = 24$$

$$36^8 \bmod 47 = 24^2 \bmod 47 = 576 \bmod 47 = 12$$

Mit diesen Quadraten gilt:

$$\begin{aligned} 36^{13} \bmod 47 &= 36^{8+4+1} \bmod 47 \\ &= 36^8 \cdot 36^4 \cdot 36^1 \bmod 47 \\ &= 12 \cdot 24 \cdot 36 \bmod 47 \\ &= 12 \cdot 864 \bmod 47 = 12 \cdot 18 \bmod 47 = 216 \bmod 47 = 28 \end{aligned}$$

Frage 17

Gegeben seien die DH-Parameter $(n, g) = (107, 2)$, bezüglich der Alice den privaten Schlüssel 67 und Bob den privaten Schlüssel 57 besitzt. Berechnen Sie die öffentlichen Schlüssel von Alice und Bob sowie ihren gemeinsamen Schlüssel $k_{A,B}$.

2.1 Exkurs: Berechnung modularer Potenzen

Es gibt zwei verschiedene Varianten zur effizienten Berechnung modularer Potenzen, die sich beide an der Darstellung des Exponenten e im Stellenwertsystem zur Basis 2 (Binärsystem) orientieren.

Ist

$$e = (e_{l-1} \dots e_2 e_1 e_0)_2 = \sum_{i=0}^{l-1} e_i \cdot 2^i \quad \text{mit } e_i \in \mathbb{Z}_2 \quad (2.1)$$

so gilt für alle Zahlen a einerseits

$$a^e = \prod_{i=0}^{l-1} (a^{2^i})^{e_i} \quad (2.2)$$

und andererseits

$$a^e = (a^{\tilde{e}})^2 \cdot a^{e_0} \quad (2.3)$$

mit:

$$\tilde{e} = \left\lfloor \frac{e}{2} \right\rfloor = (e_{l-1} \dots e_2 e_1)_2 = \sum_{i=1}^{l-1} e_i \cdot 2^{i-1} \quad (2.4)$$

Gleichung (2.2) führt zu einem Algorithmus bei dem sukzessive die Quadrate

$$a^2, \quad a^4 = (a^2)^2, \quad a^8 = (a^4)^2, \dots, \quad a^{2^{l-1}} = \left(a^{2^{l-2}}\right)^2$$

in $\mathbb{Z}_n$ berechnet und jeweils als Faktor bei der Berechnung von $p = a^e$ berücksichtigt werden, falls das entsprechende Bit e_i gleich 1 ist:

Berechnung von Potenzen in $\mathbb{Z}_n$ auf Grundlage von Gleichung (2.2)

Sei $n \in \mathbb{N}$, $a \in \mathbb{Z}_n$ und $e \in \mathbb{N}$ mit den Ziffern $e_i \in \mathbb{Z}_2$ im Stellenwertsystem zur Basis 2 (Binärsystem):

$$e = (e_{l-1} \dots e_2 e_1 e_0)_2 = \sum_{i=0}^{l-1} e_i \cdot 2^i$$

Der folgende Algorithmus liefert $p = a^e \bmod n$, wobei i , p und s als Variablen im Sinne einer Programmiersprache aufzufassen sind, die Zahlen aus $\mathbb{N}_0$ speichern können.

- (1) Setze: $i := 0$, $p := 1$ und $s := a$
- (2) Ist $e_i = 1$, so setze $p := p \cdot s \bmod n$
- (3) Ist $i = l - 1$ so beende den Algorithmus und gebe den Wert von p aus.
- (4) Setze: $i := i + 1$
- (5) Setze: $s := s^2 \bmod n$
- (6) Gehe zu (2).

Beispiel 2.4

Um die Funktionsweise des oben beschriebenen Algorithmus beispielhaft nachzuvollziehen, soll die Berechnung der bereits in den Beispielen 2.2 und 2.3 betrachteten Potenz $36^{13} \bmod 47$ nochmals detailliert in der Reihenfolge der im Algorithmus durchzuführenden Zuweisungen von Werten an die Variablen p und s (Schritte (1), (2) und (5)) durchgeführt werden. Der jeweils aktuell der Variablen p zugeordnete Wert ist dabei

unterstrichen.

$$\begin{aligned}
 & 36^{13} \bmod 47 \\
 &= 36^{1101_2} \bmod 47 \\
 &= 36^{1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0} \bmod 47 \\
 &= (36^{2^3})^1 \cdot (36^{2^2})^1 \cdot (36^{2^1})^0 \cdot (36^{2^0})^1 \bmod 47 \\
 &\stackrel{(1)}{=} (36^{2^3})^1 \cdot (36^{2^2})^1 \cdot (36^2)^0 \cdot \underline{36^1} \cdot \underline{1} \bmod 47 \quad (s = 36) \\
 &\stackrel{(2)}{=} (36^{2^3})^1 \cdot (36^{2^2})^1 \cdot (36^2)^0 \cdot \underline{36} \bmod 47 \\
 &\stackrel{(5)}{=} (36^{2^3})^1 \cdot (27^2)^1 \cdot 27^0 \cdot \underline{36} \bmod 47 \quad (s = 36^2 \bmod 47 = 27) \\
 &\stackrel{(5)}{=} (24^2)^1 \cdot 24^1 \cdot \underline{36} \bmod 47 \quad (s = 27^2 \bmod 47 = 24) \\
 &\stackrel{(2)}{=} (24^2)^1 \cdot 864 \bmod 47 = (24^2)^1 \cdot \underline{18} \bmod 47 \\
 &\stackrel{(5)}{=} 12^1 \cdot \underline{18} \bmod 47 \quad (s = 24^2 \bmod 47 = 12) \\
 &\stackrel{(2)}{=} 216 \bmod 47 = \underline{28}
 \end{aligned}$$

Gleichung (2.3) zur rekursiven Berechnung einer Potenz führt zu einem Algorithmus bei dem die Bits e_i der Binärdarstellung des Exponenten $e = (e_{l-1} \dots e_2 e_1 e_0)_2$ von links nach rechts abgearbeitet werden:

Berechnung von Potenzen in $\mathbb{Z}_n$ auf Grundlage von Gleichung (2.3)

Sei $n \in \mathbb{N}$, $a \in \mathbb{Z}_n$ und $e \in \mathbb{N}$ mit den Ziffern $e_i \in \mathbb{Z}_2$ im Stellenwertsystem zur Basis 2 (Binärsystem):

$$e = (e_{l-1} \dots e_2 e_1 e_0)_2 = \sum_{i=0}^{l-1} e_i \cdot 2^i$$

Der folgende Algorithmus liefert $p = a^e \bmod n$, wobei i und p als Variablen im Sinne einer Programmiersprache aufzufassen sind, die Zahlen aus $\mathbb{N}_0$ speichern können.

- (1) Setze: $i := l - 1$ und $p := 1$
- (2) Ist $e_i = 1$, so setze $p := p \cdot a \bmod n$
- (3) Ist $i = 0$ so beende den Algorithmus und gebe den Wert von p aus.
- (4) Setze: $i := i - 1$
- (5) Setze: $p := p^2 \bmod n$
- (6) Gehe zu (2).

Beispiel 2.5

Betrachten wir nochmals die Berechnung von $36^{13} \bmod 47$. Diesmal unter Benutzung der Rekursionsgleichung (2.3), d. h. durch eine Abarbeitung der Bits in der Binärdarstellung des Exponenten

$$13 = 1101_2$$

von links nach rechts:

$$36^{1_2} \bmod 47 = 36$$

$$\begin{aligned} 36^{11_2} \bmod 47 &= 36^2 \cdot 36 \bmod 47 \\ &= 1296 \cdot 36 \bmod 47 = 27 \cdot 36 \bmod 47 = 972 \bmod 47 \\ &= 32 \end{aligned}$$

$$\begin{aligned} 36^{110_2} \bmod 47 &= 32^2 \bmod 47 \\ &= 1024 \bmod 47 \\ &= 37 \end{aligned}$$

$$\begin{aligned} 36^{1101_2} \bmod 47 &= 37^2 \cdot 36 \bmod 47 \\ &= 1369 \cdot 36 \bmod 47 = 6 \cdot 36 \bmod 47 = 216 \bmod 47 \\ &= 28 \end{aligned}$$

Die Rechnung lässt sich auch folgendermaßen in einer einzigen Gleichungskette darstellen. Die Reihenfolge der Rechenschritte entspricht dabei der Reihenfolge der im zuletzt beschriebenen Algorithmus durchzuführenden Zuweisungen von Werten an die Variable p (Schritte (1), (2) und (5)). Der jeweils aktuell der Variablen p zugeordnete Wert ist unterstrichen:

$$\begin{aligned} 36^{13} \bmod 47 &= 36^{1101_2} \bmod 47 \\ &\stackrel{(1)}{=} \left(\left((\underline{1} \cdot 36^1)^2 \cdot 36^1 \right)^2 \cdot 36^0 \right)^2 \cdot 36^1 \bmod 47 \\ &\stackrel{(2)}{=} \left((\underline{36^2} \cdot 36)^2 \right)^2 \cdot 36 \bmod 47 \\ &\stackrel{(5)}{=} \left((1296 \cdot 36)^2 \right)^2 \cdot 36 \bmod 47 = \left((\underline{27} \cdot 36)^2 \right)^2 \cdot 36 \bmod 47 \\ &\stackrel{(2)}{=} (972^2)^2 \cdot 36 \bmod 47 = (\underline{32^2})^2 \cdot 36 \bmod 47 \\ &\stackrel{(5)}{=} 1024^2 \cdot 36 \bmod 47 = \underline{37^2} \cdot 36 \bmod 47 \\ &\stackrel{(5)}{=} 1369 \cdot 36 \bmod 47 = \underline{6} \cdot 36 \bmod 47 \\ &\stackrel{(2)}{=} 216 \bmod 47 = \underline{28} \end{aligned}$$

2.2 Exkurs: Angriffe auf das DH-Verfahren

2.2.1 Pohlig-Hellman-Reduktion

Natürlich kann das DL-Problem im Prinzip durch eine Brute-Force-Berechnung der Potenzen von g gelöst werden. Damit eine solche Brute-Force-Suche nicht praktikabel ist, muss die Ordnung von g hinreichend groß sein. In der Regel wird, wie in Beispiel 2.1, für n eine Primzahl gewählt. Dann besitzt $\mathbb{Z}_n^*$ primitive Elemente, d. h. Elemente g mit der Ordnung $o(g) = n - 1$.

Allerdings ist die Ordnung von g nicht allein ausschlaggebend, da eine Reduktion der Brute-Force-Suche auf die Untergruppen von $\langle g \rangle$ mit maximaler Primzahlpotenzordnung möglich ist. Diese *Pohlig-Hellman-Reduktion* basiert auf dem Verfahren des chinesischen Restsatzes zur Bestimmung einer simultanen Lösung von Kongruenzen bezüglich teilerfremder Zahlen n_i . Die folgende Beschreibung folgt daher dem Beweis von Satz 1.9, wobei $n_i = p_i^{e_i}$ ist und $n - 1$ anstelle von n auftritt.

Es sei n eine Primzahl und $n - 1$ besitze die Primfaktorzerlegung

$$n - 1 = p_1^{e_1} \cdots p_r^{e_r}$$

mit paarweise verschiedenen Primzahlen p_i . Weiterhin sei

$$m_i := \frac{n - 1}{p_i^{e_i}} \quad \text{für } i = 1, \dots, r$$

und $v_i \in \mathbb{Z}_{p_i^{e_i}}$ (berechenbar mit erweiterten Euklid'schen Algorithmus) mit:

$$v_i \cdot m_i \bmod p_i^{e_i} = 1 \quad \text{für } i = 1, \dots, r$$

Für

$$\tilde{a} = \sum_{i=1}^r v_i \cdot m_i$$

gelten dann simultan die Kongruenzen

$$\tilde{a} \equiv 1 \pmod{p_i^{e_i}} \quad \text{für } i = 1, \dots, r$$

und daher ist:

$$\left(\sum_{i=1}^r v_i \cdot m_i \right) = \tilde{a} \equiv 1 \pmod{n - 1} \quad (*)$$

Nach diesen Vorbereitungen kann die Pohlig-Hellman-Reduktion zur Bestimmung eines x mit

$$g^x = a$$

für gegebene $g, a \in \mathbb{Z}_n^*$ mit $o(g) = n - 1$ durchgeführt werden. Hierzu müssen für

$$g_i := g^{m_i} \quad \text{und} \quad a_i := a^{m_i}$$

Lösungen $x_i \in \mathbb{Z}_{p_i^{e_i}}$ der Gleichungen

$$g_i^{x_i} = a_i \quad \text{für } i = 1, \dots, r$$

gefunden werden. (Beachten Sie: $o(g_i) = p_i^{e_i}$.)

Sind solche Lösungen x_i gefunden, so ist das ursprünglich gesuchte x durch

$$x := \left(\sum_{i=1}^r v_i \cdot m_i \cdot x_i \right) \bmod (n - 1)$$

gegeben, da gilt:

$$\begin{aligned} g^x &= g^{\sum_{i=1}^r v_i \cdot m_i \cdot x_i} \\ &= \prod_{i=1}^r g^{v_i \cdot m_i \cdot x_i} = \prod_{i=1}^r g_i^{v_i \cdot x_i} = \prod_{i=1}^r a_i^{v_i} = \prod_{i=1}^r a^{m_i \cdot v_i} \\ &= a^{\sum_{i=1}^r m_i \cdot v_i} \stackrel{(*)}{=} a \end{aligned}$$

Bitte beachten Sie: Die Wahl einer sicheren Primzahl n , bei der $n-1$ die Primfaktorzerlegung

$$n-1 = 2 \cdot q$$

mit einer Primzahl q besitzt, stellt eine optimale Wahl zur Abwehr einer Pohlig-Hellman-Reduktion dar.

2.2.2 Shanks' „Baby Steps Giant Steps“-Verfahren (BSGS-Verfahren)

Sei $g \in \mathbb{Z}_n^*$ und $m = o(g)$. Auch für den Fall, dass m eine Primzahlpotenz ist und eine Pohlig-Hellman-Reduktion nicht möglich ist, kann eine Lösung x für die Gleichung (DL-Problem)

$$g^x = a \quad \text{für ein gegebenes } a \in \langle g \rangle$$

wesentlich schneller als mit einer naiven Brute-Force-Suche, bei der durchschnittlich $\frac{m}{2}$ viele Potenzen zu berechnen wären, gefunden werden, falls ein hinreichend großer Datenspeicher zur Verfügung steht.

Besteht die Möglichkeit

$$\tilde{m} := \lceil \sqrt{m} \rceil$$

viele Zahlen aus $\mathbb{Z}_n$ in einer Tabelle zu speichern, so lässt sich das gesuchte x mit dem BSGS-Verfahren tatsächlich bereits mit der Berechnung von höchstens $2\tilde{m}$ vielen Potenzen finden. Das Verfahren weist eine große Ähnlichkeit zu dem im Studienheft Datensicherheit 02, Abschnitt 2.6 beschriebenen Meet-in-the-middle-Angriff auf.

Im Stellenwertsystem zur Basis $\tilde{m}$ besitzt das gesuchte x eine Darstellung der Form:

$$x = x_2 \cdot \tilde{m} + x_1 \quad \text{mit } x_1, x_2 \in \{0, \dots, \tilde{m}-1\}$$

Mit dieser Darstellung gilt:

$$\begin{aligned} g^x = a &\iff g^{\tilde{m} \cdot x_2} \cdot g^{x_1} = a \\ &\iff g^{x_1} = a \cdot (g^{-\tilde{m}})^{x_2} \end{aligned} \quad (*)$$

Ein „meet in the middle“ wird hier durch ein Paar (x_1, x_2) repräsentiert, für das $(*)$ gilt und ein solches Paar kann gefunden werden, indem zunächst die Werte g^{x_1} für $x_1 = 0, \dots, \tilde{m}-1$ in einer Tabelle sortiert abgelegt werden und anschließend die Werte $a \cdot (g^{-\tilde{m}})^{x_2}$ für $x_2 = 0, 1, \dots$ solange berechnet werden, bis ein Treffer in der Tabelle gefunden wird.

2.2.3 Pollard's Rho-Methode

Sei wieder $g \in \mathbb{Z}_n^*$ und $m = o(g)$. Wie bei dem BSGS-Verfahren ist auch bei der Rho-Methode die Bestimmung einer Lösung x für die Gleichung (DL-Problem)

$$g^x = a \quad \text{für ein gegebenes } a \in \langle g \rangle$$

mit einer Berechnung von nur $O(\sqrt{m})$ vielen Potenzen möglich. Die Rho-Methode ist probabilistischer Natur, besitzt aber im Vergleich zum BSGS-Verfahren den Vorteil, keine besonderen Ansprüche an einen Speicherbereich zu haben.

Die erste Überlegung besteht darin, dass bei einer Berechnung von Zahlen der Form

$$g^{x_i} \cdot a^{y_i}$$

mit zufällig gewählten Exponenten $x_1, y_1, x_2, y_2, \dots$ aus $\mathbb{Z}_m$ nach ungefähr $O(\sqrt{m})$ vielen Berechnungen mit einer Kollision zu rechnen ist. Für eine solche Kollision gilt:

$$\begin{aligned} g^{x_i} \cdot a^{y_i} &= g^{x_j} \cdot a^{y_j} \\ \iff g^{x_i - x_j} &= a^{y_j - y_i} \end{aligned}$$

In dem sehr wahrscheinlichen Fall, dass $\text{ggT}(y_j - y_i, m) = 1$ ist, besitzt $y_j - y_i$ ein inverses Element $\tilde{y}$ in $\mathbb{Z}_m^*$. Für $x := (x_i - x_j) \cdot \tilde{y} \bmod m$ gilt dann $g^x = a$.

Die oben beschriebene Kollision könnte mithilfe eines Speicherbereichs der Größenordnung $O(\sqrt{m})$ gefunden werden. Mithilfe der Rho-Methode kann aber auf einen solchen Speicherbereich verzichtet werden. Hierzu wird zunächst anstelle der zufälligen Wahl der Exponenten x_i, y_i eine Abbildung

$$f: \mathbb{Z}_m^2 \rightarrow \mathbb{Z}_m^2$$

definiert, mit der ausgehend von einem zufällig gewählten Paar $(x_1, y_1) \in \mathbb{Z}_m^2$ die weiteren Paare rekursiv berechnet werden:

$$(x_2, y_2) = f(x_1, y_1), \quad (x_3, y_3) = f(x_2, y_2) = f^2(x_1, y_1), \quad \dots$$

Dabei soll die Abbildung f eine Selbstabbildung auf $\langle g \rangle$ induzieren, d. h. mit der Bezeichnung

$$(g, a)^{(x, y)} := g^x \cdot a^y$$

gilt:

$$(g, a)^{(x, y)} = (g, a)^{(\tilde{x}, \tilde{y})} \implies (g, a)^{f(x, y)} = (g, a)^{f(\tilde{x}, \tilde{y})} \quad (*)$$

Bei der Konstruktion von f ist darauf zu achten, dass die Folgen der Zahlen $(g, a)^{(x_i, y_i)} = g^{x_i} \cdot a^{y_i}$ für hinreichend große Bereiche Folgen von echten Zufallszahlen in $\langle g \rangle$ gut simulieren. Tritt nun in einer solchen Folge für $i = r$ und $j = r + s$ mit $s > 0$ eine erste Kollision

$$(g, a)^{(x_r, y_r)} = (g, a)^{(x_{r+s}, y_{r+s})}$$

auf, so gilt wegen (*) auch:

$$(g, a)^{(x_i, y_i)} = (g, a)^{(x_{i+s}, y_{i+s})} \quad \text{für alle } i \geq r$$

Beginnend mit dem r -ten Folgenglied wird die Folge der $(g, a)^{(x_i, y_i)}$ also periodisch mit der Periode s . Diese Periodizität erlaubt den Verzicht auf nennenswerte Speicherressourcen. Es werden lediglich jeweils zwei Tripel

$$T_i = \left(x_i, y_i, (g, a)^{(x_i, y_i)} \right)$$

gespeichert. Ausgehend von dem Tripelpaar (T_1, T_2) werden rekursiv die Paare

$$(T_i, T_{2i})$$

berechnet bis ein Paar mit einer Kollision

$$(g, a)^{(x_i, y_i)} = (g, a)^{(x_{2i}, y_{2i})}$$

gefunden wird. (Mit den obigen Bezeichnungen ist dies für das kleinste $i \in \mathbb{N}$ der Fall, für das $i \geq r$ und $s \mid i$ gilt.)

Es bleibt die Angabe einer konkreten Funktion $f : \mathbb{Z}_m^2 \rightarrow \mathbb{Z}_m^2$ mit den weiter oben geforderten Eigenschaften. Eine solche Funktion ergibt sich nach einer disjunkten Zerlegung von $\langle g \rangle$ in drei ungefähr gleich große Teilmengen

$$\langle g \rangle = G_1 \cup G_2 \cup G_3$$

in folgender Weise:

$$f : (x, y) \mapsto \begin{cases} (x+1, y) & \text{falls } (g, a)^{(x, y)} \in G_1 \\ (2x, 2y) & \text{falls } (g, a)^{(x, y)} \in G_2 \\ (x, y+1) & \text{falls } (g, a)^{(x, y)} \in G_3 \end{cases}$$

Die Aufteilung der Elemente von $\langle g \rangle \subset \mathbb{Z}_n$ in drei disjunkte Mengen ist beispielsweise folgendermaßen möglich:

$$G_i = \langle g \rangle \cap \{h \in \mathbb{Z}_n \mid h \bmod 3 = i - 1\}$$

2.2.4 Zahlkörpersieb

Zu Beginn der 1990er Jahre wurde das *Zahlkörpersieb* (*Number Field Sieve* (NFS)) als Methode zur Faktorisierung großer Zahlen n und zur Bestimmung diskreter Logarithmen in $\mathbb{Z}_n^*$ entwickelt. Die Laufzeit zur Bestimmung diskreter Logarithmen in $\mathbb{Z}_n^*$ mithilfe des Zahlkörpersiebs kann durch

$$O\left(e^{\left(\left(\frac{64}{9}\right)^{1/3} + o(1)\right)(\ln(n))^{1/3}(\ln(\ln(n)))^{2/3}}\right)$$

von oben abgeschätzt werden.

Das RSA-Verfahren

3

3.1	RSA-Schlüsselpaare	41
3.2	Das RSA-Verschlüsselungsverfahren	45
3.3	Das RSA-Signaturverfahren	51

Das nach den Initialen ihrer Erfinder R. Rivest, A. Shamir und L. Adleman benannte *RSA-Verfahren* kann sowohl als asymmetrisches Verschlüsselungsverfahren als auch als Signaturverfahren eingesetzt werden.

Grundlegend für die Funktionsweise des RSA-Verfahrens ist die folgende Implikation des kleinen Satzes von Fermat (Satz 1.6):

Satz 3.1

Es seien p und q zwei verschiedene Primzahlen und:

$$n := p \cdot q$$

Weiterhin sei $r \in \mathbb{Z}$ mit

$$r \equiv 1 \pmod{p-1} \quad (3.1)$$

und:

$$r \equiv 1 \pmod{q-1} \quad (3.2)$$

Dies gilt zum Beispiel für $r = 1 + (p-1)(q-1)$ oder $r = 1 + \text{kgV}(p-1, q-1)$.

Dann gilt für alle $z \in \mathbb{Z}_n$:

$$z^r = z \quad (3.3)$$

Beweis Wir zeigen zuerst, dass für alle $z \in \mathbb{Z}$ gilt:

$$p \mid (z^r - z) \quad (*)$$

Selbstverständlich gilt (*), falls p ein Teiler von z ist. Gelte nun $p \nmid z$.

Wegen (3.1) gilt $(p-1) \mid (r-1)$ und es gibt ein $s \in \mathbb{N}$ mit:

$$\begin{aligned} r-1 &= (p-1) \cdot s \\ \implies r &= 1 + (p-1) \cdot s \end{aligned}$$

Mit dem kleinen Satz von Fermat (Satz 1.6) folgt (*) nun wegen:

$$z^r = z \cdot (z^{p-1})^s \equiv z \cdot 1^s = z \pmod{p}$$

Genauso folgt mit (3.2), dass für alle $z \in \mathbb{Z}$ gilt:

$$q \mid (z^r - z) \quad (**)$$

Da p und q verschiedene Primzahlen sind, folgt aus (*) und (**), dass $z^r - z$ auch von $n = p \cdot q$ geteilt wird. Für alle $z \in \mathbb{Z}$ gilt also

$$z^r \equiv z \pmod{n}$$

und somit ist auch die äquivalente Gleichung (3.3) für $z \in \mathbb{Z}_n$ bewiesen. ■

Frage 18

Zeigen Sie, dass $r \in \mathbb{Z}$ genau dann die Bedingungen (3.1) und (3.2) erfüllt, falls gilt:

$$r \equiv 1 \pmod{\text{kgV}(p-1, q-1)} \quad (3.4)$$

3.1 RSA-Schlüsselpaare

Unter Verwendung der in Satz 3.1 eingeführten Bezeichnungen kann die Konstruktion eines RSA-Schlüsselpaars beschrieben werden:

RSA-Schlüsselpaar

Alice generiert ihr Schlüsselpaar folgendermaßen:

- Alice bestimmt zwei verschiedene Primzahlen

$$p \quad \text{und} \quad q$$

mit einem Zufallszahlengenerator. (Die Primzahlen besitzen beispielsweise 2048 Bitstellen.) Alice hält die Werte von p und q geheim.

- Alice berechnet:

$$n = p \cdot q$$

- Alice berechnet

$$v = \text{kgV}(p-1, q-1)$$

und hält den Wert von v geheim.

Alternativ kann Alice auch ein Vielfaches von $\text{kgV}(p-1, q-1)$ für v wählen. Entscheidend ist, dass $r = 1 + v$ die Bedingungen (3.1) und (3.2) erfüllt. Beliebige auch die Verwendung von $v = (p-1)(q-1)$.

- Alice wählt eine Zahl:

$$e \in \mathbb{Z}_v^*$$

Die Zahl e wird als *öffentlicher Exponent* des Schlüsselpaars bezeichnet. (Der Grund hierfür wird bei den Anwendungen des Schlüssels offensichtlich.)

- Alice bestimmt die Inverse zu e in $\mathbb{Z}_v^*$:

$$d = e^{-1} \in \mathbb{Z}_v^*$$

Die Zahl d wird als *privater Exponent* des Schlüsselpaars bezeichnet.

- Der öffentliche Schlüssel von Alice besteht aus dem Zahlenpaar:

$$k_{A, \text{pub}} = (n, e)$$

- Der private Schlüssel von Alice besteht aus dem Zahlenpaar:

$$k_{A, \text{priv}} = (n, d)$$

Beispiel 3.1

RSA-Schlüsselpaare können innerhalb eines Java-Programms mit einer geeigneten Instanz der Klasse `KeyPairGenerator` erzeugt werden. (Siehe hierzu den in der Abbildung 3.1 wiedergegebene Quellcode.)

Beispielsweise wurde folgendes Schlüsselpaar mit einer Bitlänge $\lceil \log_2(n) \rceil = 2048$ auf diese Weise erzeugt:

```

n = 16168071105410691789333727253545285425054602153245
    90204154619291826180517268289289260262961391645385
    87252292573042195868358338780212272813328285432930
    98938867283088253785506450582562311172139885613039
    47832745852869850074523674809445697461898848258982
    56177917054178929148242979069451204216293772563056
    69543996313975448783597681650671949525006213463201
    70488387186763666227169881155057139124481245006496
    90477201599126274576409287155040493852180020698941
    33852393880073215037370387712463302102903103361251
    39476232741977840751451359061623686793523945503881
    07459238974533011939826218652756330708024526384519
    82798725355045671

```

```
e = 65537
```

```

d = 12935048967741785432631124210108586072088482843263
    9445785182414069960933375207043570717420566805787
    14417996005153553164926139721441168929740889296419
    39111687892135485641418957488821690028192295687365
    70071357104500846531080539506063091220566709063810
    88055915696243490136879562362779277957012299663185
    50826720804422302159550428738857532165935936434657
    09409990352714686398188649999443761602361578163048
    17343088468413646273406481388390630608925889022555
    01537512298374597901048595576433506634785244042404
    20343235304253602775533150811070033763661398799934
    67055475814167460324061409325201045135140539718680
    44161275601117057

```

```

1  KeyPairGenerator kpgenerator
    = KeyPairGenerator.getInstance("RSA");
    kpgenerator.initialize(2048);
    KeyPair rsaKeyPair = kpgenerator.generateKeyPair();

5  RSAPublicKey publicKey
    = (RSAPublicKey) rsaKeyPair.getPublic();
    RSAPrivateKey privateKey
    = (RSAPrivateKey) rsaKeyPair.getPrivate();

10 BigInteger n = publicKey.getModulus();
    BigInteger e = publicKey.getPublicExponent();
    BigInteger d = privateKey.getPrivateExponent();

```

Abb. 3.1 Erzeugung eines RSA-Schlüsselpaars in einem Java-Programm

Kommentar Sehr häufig wird wie in obigen Beispiel 3.1 für den öffentlichen Exponenten e die Fermat'sche¹ Primzahl

$$F_4 = 2^{2^4} + 1 = 2^{16} + 1 = 65537$$

gewählt. Damit e in $\mathbb{Z}_v^*$ liegt, darf $e = F_4$ weder ein Teiler von $p - 1$ noch von $q - 1$ sein. Diese Bedingungen müssen dann natürlich bei der Zufallswahl von p und q berücksichtigt werden. ◀

Kommentar Im ersten Schritt der RSA-Schlüsselpaarerzeugung muss Alice zwei Primzahlen aus einer hinreichend großen Teilmenge der natürlichen Zahlen zufällig auswählen, sodass das Produkt der beiden Primzahlen eine Zahl mit einer vorgegebenen Bitlänge ergibt.

Hierzu wird neben einem guten (kryptographisch sicheren) Zufallszahlengenerator ein Algorithmus benötigt, mit dem entschieden werden kann, ob eine gegebene Zahl eine Primzahl ist. Hier kommen wiederum die in dem Kommentar auf Seite 27 erwähnten probabilistischen Primzahltests zum Einsatz. ◀

Beispiel 3.2

Realistische Beispiele von RSA-Schlüsselpaaren mit Zahlen, die mehrere hundert Dezimalstellen besitzen, können natürlich nur mit einem Computer bearbeitet werden.

Die durchzuführenden Rechnungen lassen sich aber analog zu entsprechenden Beispielen für das DH-Verfahren (wie in Beispiel 2.2) mit kleinen Zahlen nachvollziehen. Beispielsweise berechnet Alice ausgehend von den Primzahlen

$$p = 41 \quad \text{und} \quad q = 43$$

die Zahlen

$$n = p \cdot q = 1763$$

$$v = (p - 1)(q - 1) = 40 \cdot 42 = 1680$$

und wählt die zu v teilerfremde Zahl

$$e = 11 \in \mathbb{Z}_v^*$$

als öffentlichen Exponenten.

Den privaten Exponenten d berechnet Alice mithilfe des erweiterten Euklid'schen Algorithmus:

1680	11		q
1	0	1680	
0	1	11	152
1	-152	8	1
-1	153	3	2
3	-458	2	1
-4	611	1	

Es ist also

$$d = e^{-1} = 611 \in \mathbb{Z}_v^*$$

und das Schlüsselpaar besteht aus den Schlüsseln:

$$k_{A, \text{pub}} = (n, e) = (1763, 11)$$

$$k_{A, \text{priv}} = (n, d) = (1763, 611)$$

¹ Pierre de Fermat (*1607 Beaumont-de-Lomagne, †1665 Castres)

Bevor die Anwendungen mit einem RSA-Schlüsselpaar behandelt werden, sollten wir uns davon überzeugen, dass es bei Kenntnis des öffentlichen Schlüssels nicht möglich ist, den privaten Schlüssel, d. h. den Wert des privaten Exponenten d zu bestimmen. Dies hängt entscheidend davon ab, dass eine Faktorisierung von n in seine Primteiler nicht effizient möglich ist:

Faktorisierungs-Problem (Integer Factorization Problem, IF-Problem)

Gegeben sei eine natürliche Zahl $n \in \mathbb{N}$, die mindestens zwei große Primteiler p und q mit $p \neq q$ besitzt. Die Aufgabenstellung, allein aus der Kenntnis von n die Primteiler von n zu finden, wird als Faktorisierungs-Problem (IF-Problem) bezeichnet.

Bisher ist kein Algorithmus bekannt, der das IF-Problem effizient löst. Allerdings gibt es auch keinen Beweis, dass es einen solchen Algorithmus nicht geben kann.

Könnte ein Angreifer das IF-Problem für die in dem RSA-Verfahren genutzte Zahl $n = p \cdot q$ lösen, so könnte er auch mit den gleichen Berechnungen, mit denen Alice den privaten Exponenten d bestimmt hat, diesen ebenfalls bestimmen. Das RSA-Verfahren kann also nur solange als ein sinnvolles asymmetrisches Verfahren betrachtet werden, solange das Faktorisierungs-Problem nicht effizient gelöst werden kann.

Andererseits gibt es keinen effizienteren Weg als die Lösung des IF-Problems für n , um den privaten Schlüssel allein aus der Kenntnis des öffentlichen Schlüssels zu bestimmen. Ist nämlich der private Exponent d bekannt, so kann die Zahl

$$v = d \cdot e - 1$$

berechnet werden, die ein Vielfaches von $\text{kgV}(p-1, q-1)$ ist. Ausgehend von v lassen sich dann mit dem in Satz 1.13 beschriebenen Algorithmus die Primfaktoren p und q effizient bestimmen.

Frage 19

Gegeben seien die Zahlen $p, q \in \mathbb{N}$ mit den Bitlängen $l_2(p)$ und $l_2(q)$. Welche Bitlänge hat das Produkt pq ?

Frage 20

Für die Bitlängen der Zahlen $p, q \in \mathbb{N}$ gelte:

$$l = l_2(p) = l_2(q)$$

Im Dualsystem besitze p die folgende Darstellung:

$$p = 1 \underbrace{0 \dots 0}_m 1 * \dots *$$

D. h., in der Binärdarstellung von p treten mindestens zwei Stellen mit dem Wert 1 auf und zwischen den beiden höchstwertigen solchen Stellen liegen m Stellen, deren Wert 0 ist ($m \in \mathbb{N}_0$). Die Ziffern, die den ersten $m+2$ Ziffern folgen, sind beliebig.

Weiterhin seien in der Binärdarstellung von q die $m+2$ höchstwertigen Bits alle 1:

$$q = \underbrace{1 \dots 1}_{m+2} * \dots *$$

Zeigen Sie, dass unter diesen Voraussetzungen gilt:

$$l_2(pq) = 2l$$

Kommentar Insbesondere folgt mit dem Ergebnis aus Frage 20, dass das Produkt von zwei natürlichen Zahlen p, q mit $l = l_2(p) = l_2(q)$ die Bitlänge $l_2(pq) = 2l$ besitzt, falls die beiden höchstwertigen Bitstellen von p und von q jeweils 1 sind. ◀

3.2 Das RSA-Verschlüsselungsverfahren

RSA-Verschlüsselung

Bob möchte eine Nachricht m verschlüsselt an Alice schicken.

- Zunächst benötigt Bob den öffentlichen Schlüssel

$$k_{A, \text{pub}} = (n, e)$$

von Alice. (Dieser könnte beispielsweise zuvor von Alice per Email an Bob geschickt werden.)

- Die zu verschlüsselnde Nachricht m sei durch eine Bitfolge repräsentiert. Dieser wiederum entspricht eine nicht-negative ganze Zahl im Binärsystem. Diese Zahl wird im Folgenden ebenfalls mit m bezeichnet und es wird außerdem angenommen, dass

$$m < n$$

gilt, d. h., dass $m \in \mathbb{Z}_n$ ist.

- Bob berechnet die Geheimtextnachricht

$$c = E_{k_{A, \text{pub}}}(m) := m^e \bmod n \quad (3.5)$$

und schickt diese an Alice über einen möglicherweise unsicheren Kanal.

- Nach dem Empfang von c berechnet Alice mithilfe ihres privaten Schlüssels

$$k_{A, \text{priv}} = (n, d)$$

die Zahl:

$$\tilde{m} = D_{k_{A, \text{priv}}}(c) := c^d \bmod n \quad (3.6)$$

Tatsächlich liefert $D_{k_{A, \text{priv}}}$ das zu $E_{k_{A, \text{pub}}}$ korrespondierende Entschlüsselungsverfahren, d. h. es gilt $\tilde{m} = m$:

Nach Konstruktion eines RSA-Schlüsselpaars gilt nämlich

$$e \cdot d \equiv 1 \pmod{\varphi(n)}$$

und $r = e \cdot d$ erfüllt daher die Voraussetzungen (3.1) und (3.2) aus Satz 3.1, sodass mit Satz 3.1 folgt:

$$\tilde{m} = c^d \bmod n = (m^e)^d \bmod n = m^{e \cdot d} \bmod n \stackrel{(3.3)}{=} m \quad (3.7)$$

Beispiel 3.3

Mit dem Schlüsselpaar aus dem „Schulbuch“-Beispiel 3.2

$$k_{A, pub} = (n, e) = (1763, 11)$$

$$k_{A, priv} = (n, d) = (1763, 611)$$

würde Bob beispielsweise die „Nachricht“ $m = 1234$ wie folgt verschlüsseln:

$$\begin{aligned} c &= 1234^{11} \bmod 1763 = 1234^{10} \cdot 1234 \bmod 1763 \\ &= 1287^5 \cdot 1234 \bmod 1763 \\ &= 1287^4 \cdot 1287 \cdot 1234 \bmod 1763 \\ &= 912^2 \cdot 1458 \bmod 1763 \\ &= 1371 \cdot 1458 \bmod 1763 \\ &= 1439 \end{aligned}$$

Mit ihrem privaten Schlüssel kann Alice c wieder entschlüsseln:

$$\begin{aligned} 1439^{611} \bmod 1763 &= 1439^{610} \cdot 1439 \bmod 1763 \\ &= 959^{305} \cdot 1439 \bmod 1763 \\ &= 959^{304} \cdot 959 \cdot 1439 \bmod 1763 \\ &= 1158^{152} \cdot 1335 \bmod 1763 \\ &= 1084^{76} \cdot 1335 \bmod 1763 \\ &= 898^{38} \cdot 1335 \bmod 1763 \\ &= 713^{19} \cdot 1335 \bmod 1763 \\ &= 713^{18} \cdot 713 \cdot 1335 \bmod 1763 \\ &= 625^9 \cdot 1598 \bmod 1763 \\ &= 625^8 \cdot 625 \cdot 1598 \bmod 1763 \\ &= 1002^4 \cdot 892 \bmod 1763 \\ &= 857^2 \cdot 892 \bmod 1763 \\ &= 1041 \cdot 892 \bmod 1763 \\ &= 1234 = m \end{aligned}$$



Frage 21

Zeigen Sie: Sind $n_1, n_2 \in \mathbb{N}$ mit $n_1, n_2 \geq 2$ und $\text{ggT}(n_1, n_2) = 1$, so gilt für $n = n_1 \cdot n_2$:

$$\mu_n = \text{kgV}(\mu_{n_1}, \mu_{n_2})$$

Wird das RSA-Verschlüsselungsverfahren wie in (3.5) beschrieben direkt auf eine Klartextnachricht m angewandt, so sind die bereits beim ECB-Modus von symmetrischen Blockverschlüsselungsverfahren besprochenen Gefährdungen durch Wörterbuchangriffe zu beachten.

Darüber hinaus kann eine Angreiferin Eve die Klartextnachricht m zu einer erlauschten Geheimtextnachricht c finden, falls sie weiß (oder richtiger Weise rät), dass m ein Element einer kleinen Menge M möglicher Nachrichten ist. In diesem Fall kann Eve nämlich einfach die Nachrichten aus M nacheinander mit dem öffentlich bekannten Schlüssel verschlüsseln und jeweils überprüfen, ob der berechnete Geheimtext mit c übereinstimmt.

Als Gegenmaßnahme zum Schutz vor den oben beschriebenen Angriffsmöglichkeiten wird eine Klartextnachricht vor der Durchführung der RSA-Operation mit einer hinreichend großen Zufallszahl erweitert (durch *Padding*).

Beispiel 3.4 (Padding nach PKCS #1, Version 1)

Zur Vermarktung ihres Algorithmus gründeten R. Rivest, A. Shamir und L. Adleman 1982 die Firma *RSA Data Security, Inc.* Die *RSA Laboratories*, eine Abteilung der *RSA Data Security, Inc.*, veröffentlichte seit Beginn der 1990er Jahre eine Reihe von *Public Key Cryptography Standards* (PKCS).

Der erste dieser Standards, *PKCS #1: RSA Encryption Standard*, wurde in einer ersten Version zuerst im Jahr 1991 veröffentlicht. In PKCS #1 wird ein Verfahren zur Formatierung von zu verschlüsselnden Daten spezifiziert. Mit der Notation aus der Version 2.2 des Standards, der auch als RFC 8017 (PKCS #1: RSA Cryptography Specifications Version 2.2) verfügbar ist, wird diese Formatierung als EME-PKCS1-v1_5-Kodierung bezeichnet.

Hierbei wird eine Nachricht, die als Byte-Array M eine Länge

$$mLen \leq k - 11$$

besitzt, wobei

$$k := \lfloor \log_2(n)/8 \rfloor + 1$$

die Anzahl der zur Darstellung des RSA-Moduls n benötigten Bytestellen bezeichnet, mit drei fest spezifizierten Bytewerten und einem Byte-Array PS , welches aus mindestens acht Zufallsbytes $\neq 00$ besteht, folgendermaßen erweitert:

$$EM = 0x00 \parallel 0x02 \parallel PS \parallel 0x00 \parallel M$$

Die dem Byte-Array EM entsprechende Zahl m wird dann entsprechend (3.5) verschlüsselt. Da das erste Byte von EM den Wert 00 hat, ist sichergestellt, dass $m < n$ ist. Der Wert 02 des zweiten Bytes steht für den Typ der hier beschriebenen Kodierung.

Das dem Byte-Array PS folgende Byte mit dem Wert 00 signalisiert das Ende des Paddings und erlaubt nach einer RSA-Entschlüsselung gemäß (3.6) die Identifikation des Beginns der eigentlichen Nachricht M .

Die RSA-Verschlüsselung einer Nachricht unter Anwendung des EME-PKCS1-v1_5-Paddings lässt sich, wie in Abbildung 3.2 dargestellt, in einem Java-Programm bequem ausführen. ◀

```

1 Cipher rsaCipher = Cipher.getInstance("RSA/ECB/PKCS1Padding");
  rsaCipher.init(Cipher.ENCRYPT_MODE, publicKey);
  byte[] ciphertext = rsaCipher.doFinal(message);

5  rsaCipher = Cipher.getInstance("RSA/ECB/NoPadding");
  rsaCipher.init(Cipher.DECRYPT_MODE, privateKey);
  System.out.println(Dump.dump(rsaCipher.doFinal(ciphertext)));

```

Abb. 3.2 RSA-Verschlüsselung mit PKCS1-v1_5-Padding und RSA-Entschlüsselung ohne Entfernung des Paddings

Frage 22

Die Ausgabe in der letzten Zeile des in Abbildung 3.2 wiedergegebenen Programmfragments lieferte:

```

000000 00 02 4B A9 06 5C A2 BA EC 41 D7 6B 7E 38 E0 66
000010 47 BC A8 E4 2E 18 CB 6F 2D F3 78 45 97 31 89 62
000020 66 21 AA 2C 5F 33 AE 7B AA 8C 81 44 D0 C8 65 D7
000030 14 C1 C3 F7 8C 24 89 67 84 8C 68 BE 68 E2 45 5C
000040 48 DE 5D 3C 64 A1 94 3D FC 77 9C 1A 8A BB 51 AE
000050 FD 9F 7B 8E 03 66 BE 48 B3 75 A5 BF 84 49 CD C5
000060 F0 0A FA C1 CE E2 96 04 19 6D 87 C7 20 D3 37 E1
000070 42 0B 84 2F 26 47 49 06 2E 86 7A CF 4E 17 F1 3D
000080 35 3B F7 10 D1 29 54 58 99 2F 96 5E 48 16 24 D1
000090 2E 1E C4 AE 57 D5 DA 53 34 DF 27 49 04 C5 35 D6
0000a0 35 A8 E7 43 D6 DE 3F 34 BA A8 9F 21 F4 D0 12 26
0000b0 20 E4 DE 89 A4 CC 61 47 E0 AE 5E 08 9D 0F BE 49
0000c0 22 FB 15 B4 5A BD E4 2A 2E FE 34 E9 F6 89 EA 45
0000d0 0A B1 7C C6 CF 99 72 5E BC 88 B6 EF F1 3B 42 20
0000e0 A8 8F CC 4F DA 85 D9 CA C7 0B 45 BC 44 3F F4 2D
0000f0 03 CC 71 98 19 00 53 69 6D 73 61 6C 61 62 69 6D

```

Wie lautet der Inhalt des in der Variablen `message` gespeicherten ASCII-kodierten Textes, der in Zeile 3 verschlüsselt wurde?

Beispiel 3.5 (Bleichenbacher-Angriff)

Im Jahr 1998 erregte die Veröffentlichung eines Angriffs auf RSA-verschlüsselte Nachrichten durch D. Bleichenbacher² große Aufmerksamkeit. Der *Bleichenbacher-Angriff* ist ein adaptiver Chosen-Ciphertext-Angriff auf RSA-verschlüsselte PKCS1-v1_5-kodierte Nachrichten.

Voraussetzung für die Durchführung des Angriffs ist das Vorliegen eines *Orakels*, welches einen Angreifer für jede vorgelegte Zahl $\tilde{c} < n$ darüber informiert, ob

$$\tilde{m} = \tilde{c}^d \bmod n$$

das Format einer PKCS1-v1_5-kodierten Nachricht aufweist. Der Angreifer kann dann nämlich Zahlen s finden, so dass für $\tilde{c} = s^e c \bmod n$ gilt:

$$s \cdot m \bmod n = s \cdot c^d \bmod n = \tilde{c}^d \bmod n = 00\ 02\ \dots$$

Diese Beobachtung bildet die Grundlage des Bleichenbacher-Angriffs zur Bestimmung von m , bei dem ungefähr 10^6 Anfragen an das Orakel gestellt werden müssen.

In der Praxis stellt ein Server ein Orakel in dem oben beschriebenen Sinne dar, wenn dieser den Absender einer Nachricht $\tilde{c}$ darüber informiert (absichtlich durch eine spezifische Fehlermeldung oder unabsichtlich durch ein zeitlich unterscheidbares Antwortverhalten), ob $\tilde{c}^d \bmod n$ das Format einer PKCS1-v1_5-kodierten Nachricht aufweist. 

Beispiel 3.6 (Optimal Asymmetric Encryption Padding (OAEP))

Zum Schutz gegen den in Beispiel 3.5 beschriebenen Bleichenbacher-Angriff wurde von M. Bellare und P. Rogaway in einer im Jahre 1995 publizierten Arbeit das *Optimal Asymmetric Encryption Padding* (OAEP) vorgeschlagen, welches in der zweiten Version des Standards *PKCS #1* aufgenommen und bevorzugt zur Anwendung empfohlen wurde. In RFC 8017 (PKCS #1: RSA Cryptography Specifications Version 2.2) wird die Kodierung von Nachrichten mithilfe des OAEP-Verfahrens als EME-OAEP-Kodierung bezeichnet.

Das OAEP-Verfahren verwendet eine sogenannte *Mask Generating Function*:

$$\text{MGF}(\text{mgfSeed}, \text{maskLen})$$

In Abhängigkeit von dem *Seed*-Wert mgfSeed liefert die Funktion ein pseudozufälliges Byte-Array der Länge maskLen , welches zur Maskierung (mittels XOR-Operation) der Werte eines Byte-Arrays der Länge maskLen genutzt werden kann. In RFC 8017 (PKCS #1: RSA Cryptography Specifications Version 2.2) wird hierzu eine als *MGF1* bezeichnete Funktion spezifiziert, die ihrerseits eine Hashfunktion h (per Default SHA-1) nutzt. Die von *MGF1* erzeugten pseudozufälligen Bytes sind:

$$h(\text{Seed} || 00000000) || h(\text{Seed} || 00000001) || h(\text{Seed} || 00000002) || \dots$$

Das OAEP-Verfahren verwendet darüber hinaus ein konstantes Byte-Array lHash , welches der Hashwert eines sogenannten *Labels* L , berechnet mit einer Hashfunktion $\tilde{h}$, ist:

$$\text{lHash} = \tilde{h}(L) \quad \text{mit der Bytelänge } hLen$$

Per Default ist $\tilde{h}$ wiederum SHA-1 und der Label ein leeres Byte-Array, d. h. ein Array der Länge 0.

Werden für alle Algorithmen und Werte die Defaults genutzt, so gilt:

$$\begin{aligned} h = \tilde{h} &= \text{SHA-1} \\ \text{lHash} &= \text{SHA-1}([]) \\ hLen &= 20 \end{aligned}$$

Eine Nachricht, die als Byte-Array M eine Länge

$$mLen \leq k - 2 \cdot hLen - 2$$

besitzt, wobei

$$k := \lfloor \log_2(n)/8 \rfloor + 1$$

die Anzahl der zur Darstellung des RSA-Moduls n benötigten Bytes bezeichnet, wird folgendermaßen EME-OAEP-kodiert:

² Daniel Bleichenbacher (*1964 Langenthal)

- (1) Die Nachricht M wird mit 1Hash , $k - m\text{Len} - 2h\text{Len} - 2$ vielen Nullbytes 00 und einem Byte mit dem Wert 01 zum sogenannten Datenblock DB erweitert:

$$DB = 1\text{Hash} \parallel 00 \dots 00 \parallel 01 \parallel M$$

Beachte: Die Bytelänge von DB ist $k - h\text{Len} - 1$.

- (2) Ein Byte-Array seed der Länge $h\text{Len}$ wird mit Zufallswerten gefüllt.
 (3) Berechnet werden nun

$$\text{maskedDB} = DB \oplus \text{MGF}(\text{seed}, k - h\text{Len} - 1)$$

und:

$$\text{maskedSeed} = \text{seed} \oplus \text{MGF}(\text{maskedDB}, h\text{Len})$$

- (4) Die EME-OAEP-Kodierung von M für den durch seed gegebenen Zufallswert setzt sich aus einem Nullbyte und den in (3) berechneten Byte-Arrays zusammen:

$$EM = 00 \parallel \text{maskedSeed} \parallel \text{maskedDB}$$

Die EME-OAEP-Kodierung wird in RFC 8017 (PKCS #1: RSA Cryptography Specifications Version 2.2) wie in Abbildung 3.3 dargestellt schematisch zusammengefasst, wobei die eingefügten Nullbytes in DB als *Padding String* PS bezeichnet werden.

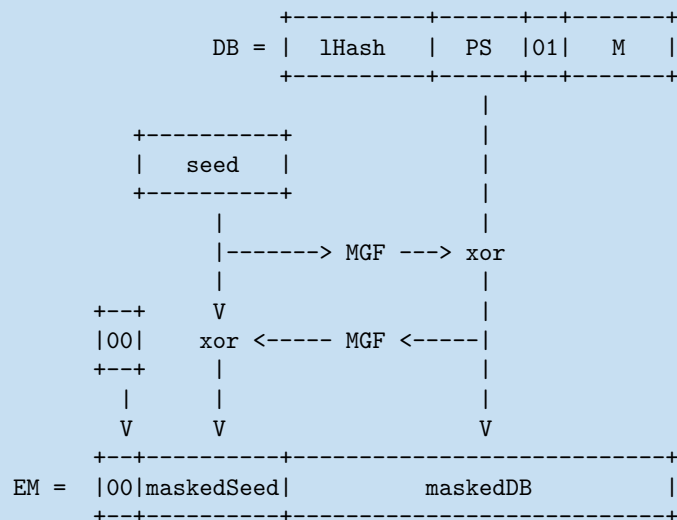


Abb. 3.3 EME-OAEP-Kodierung

Die dem Byte-Array EM entsprechende Zahl m wird entsprechend (3.5) verschlüsselt. Da das erste Byte von EM den Wert 00 hat, ist sichergestellt, dass $m < n$ ist.

Die RSA-Verschlüsselung einer Nachricht unter Anwendung der EME-OAEP-Kodierung lässt sich, wie in Abbildung 3.4 dargestellt, in einem Java-Programm bequem ausführen. ▶


```

1 Cipher rsaCipher = Cipher.getInstance("RSA/ECB/OAEPADDING");
  rsaCipher.init(Cipher.ENCRYPT_MODE, publicKey);
  byte[] ciphertext = rsaCipher.doFinal(message);

5  rsaCipher = Cipher.getInstance("RSA/ECB/NoPadding");
  rsaCipher.init(Cipher.DECRYPT_MODE, privateKey);
  byte[] em = rsaCipher.doFinal(ciphertext);
  System.out.println(Dump.dump(em));

```

Abb. 3.4 RSA-Verschlüsselung mit OAEP-Kodierung und RSA-Entschlüsselung ohne OAEP-Dekodierung

Frage 23

Die Ausgabe in der letzten Zeile des in Abbildung 3.4 wiedergegebenen Programmfragments lieferte:

```

000000 00 5C CC A0 A8 52 B4 11 7D 52 31 CD 70 0E 8F E9
000010 4D 16 17 B5 1E FE 39 AE 3D 72 E5 10 F5 75 40 AC
000020 6F D6 90 5F E2 5C 68 6F 86 85 9E 9A 73 86 37 3D
000030 1E 2D E9 20 EB C6 81 12 CE 57 EB 7E 7E 18 4B 8F
000040 BC 55 70 E2 BB B8 6D 61 AE 33 67 56 DA 32 00 F0
000050 19 1C 37 6D 7F 9E BD 7C 22 42 4C 5D B6 0F CE 43
000060 FC 3F BF 02 18 2C 42 22 69 E6 43 96 9B 97 9B 9F
000070 D7 83 47 CA FB 4D 94 A0 A1 82 0E 1D 85 10 74 C7
000080 46 66 65 0A 49 C1 EA 76 51 14 43 1E B5 1E 0F F0
000090 7A 2E DB D8 7D B2 40 D9 89 6F 7B 75 A8 DF B8 47
0000a0 95 41 6B D2 9E 71 BD 0D BB 37 45 41 CD B0 F3 03
0000b0 0D FC F0 BE 9A 7B 7E DE B8 B3 56 18 3A F9 17 51
0000c0 49 0C 33 F8 56 C6 25 7A 11 9A 51 A3 2B 7B 3F 07
0000d0 6A 73 EB 24 7C 68 77 33 BF 1F DB CB 5B D2 46 7D
0000e0 A1 D1 43 00 74 CB FF 55 68 90 C4 94 2D 8A C9 D4
0000f0 86 85 62 CA 40 B5 7D AE 2F FD CA 11 65 A6 21 E0

```

Wie lautet der Inhalt des in der Variablen `message` gespeicherten ASCII-kodierten Textes, der in Zeile 3 verschlüsselt wurde?

3.3 Das RSA-Signaturverfahren

Mit den Bezeichnungen aus den vorherigen Abschnitten nehmen wir weiterhin an, dass Alice ein RSA-Schlüsselpaar mit dem privaten Schlüssel

$$k_{A,priv} = (n, d)$$

und dem dazugehörigen öffentlichen Schlüssel

$$k_{A,pub} = (n, e)$$

besitzt.

Grundlegend für die Ver- und Entschlüsselung mit dem RSA-Verfahren ist die Gültigkeit der Gleichung in (3.7):

$$(m^e \bmod n)^d \bmod n = m^{e \cdot d} \bmod n = m \quad \text{für alle } m \in \mathbb{Z}_n$$

Ebenso gilt natürlich auch:

$$(m^d \bmod n)^e \bmod n = m^{d \cdot e} \bmod n = m \quad \text{für alle } m \in \mathbb{Z}_n \quad (3.8)$$

Ist m gegeben, so sollte nur Alice den Wert

$$s := m^d \bmod n \quad (3.9)$$

berechnen können, da hierzu der private Schlüssel benötigt wird. Ob Alice allerdings s entsprechend (3.9) durch Anwendung ihres privaten Schlüssels auf m berechnet hat, lässt sich allein mit Kenntnis des öffentlichen Schlüssels auf Grundlage von (3.8) überprüfen. Diese Beobachtung führt zum RSA-Signaturverfahren:

RSA-Signaturverfahren

Alice möchte eine Nachricht m an Bob schicken, sodass sich Bob von der Authentizität von m überzeugen kann, und Bob die Authentizität von m gegebenenfalls auch Dritten gegenüber nachweisen kann.

- Zunächst benötigt Bob den öffentlichen Schlüssel

$$k_{A, \text{pub}} = (n, e)$$

von Alice.

- Die der Nachricht m entsprechende Bitfolge wird als nicht-negative ganze Zahl (im Binärsystem) interpretiert, die ebenfalls mit m bezeichnet wird.

Es wird im Folgenden angenommen, dass

$$m < n$$

gilt, d. h., dass $m \in \mathbb{Z}_n$ ist.

- Alice berechnet eine Signatur für m entsprechend (3.9):

$$s = S_{k_{A, \text{pri}}}(m) := m^d \bmod n \quad (3.10)$$

und schickt die Nachricht m zusammen mit der Signatur s über einen möglicherweise unsicheren Kanal an Bob.

- Nach dem Empfang von m und s berechnet Bob mithilfe des öffentlichen Schlüssels von Alice die Zahl

$$\tilde{m} = s^e \bmod n$$

und überprüft die Gültigkeit von

$$\tilde{m} = m \quad (3.11)$$

Ist (3.11) erfüllt, so kann sich Bob sicher sein, dass die Nachricht m tatsächlich von Alice in exakt der vorliegenden Form erstellt wurde.

Kommentar Da im Rahmen der Signaturüberprüfung die komplette Nachricht aus der Signatur zurückgewonnen wird, ist das oben beschriebene Signaturverfahren ein sogenanntes *Signature Scheme with Message Recovery*. ◀

Wichtig Die in der Beschreibung des RSA-Signaturverfahrens gemachte Annahme, dass die der Nachricht entsprechende Zahl m kleiner als n sein soll, ist natürlich eine sehr große Einschränkung.

Um Nachrichten von (praktisch) beliebiger Länger signieren zu können, wird daher in der Regel zunächst ein Hashwert einer Nachricht berechnet und dieser dann geeignet als Zahl $< n$ kodiert, bevor schließlich das oben beschriebene RSA-Signaturverfahren angewendet wird.

Solche Verfahren sind in RFC 8017 (PKCS #1: RSA Cryptography Specifications Version 2.2) spezifiziert und werden in den folgenden Beispielen erläutert.

Aus der Signatur lässt sich bei einem solchen Verfahren die Nachricht natürlich nicht mehr zurückgewinnen. Ein Verfahren dieser Art wird als *Signature Scheme with Appendix* bezeichnet.



Beispiel 3.7 (Signature Scheme with Appendix RSASSA-PKCS1-v1_5)

In RFC 8017 (PKCS #1: RSA Cryptography Specifications Version 2.2) werden zwei verschiedene Signature Schemes with Appendix spezifiziert. Das einfachere und bereits in der ersten Version des Standards PKCS #1 standardisierte Verfahren RSASSA-PKCS1-v1_5 soll in diesem Beispiel betrachtet werden. Signaturen entsprechend RSASSA-PKCS1-v1_5 werden auch heutzutage noch vielfach im Zusammenhang mit digitalen Zertifikaten genutzt.

Die Anwendung des Verfahrens RSASSA-PKCS1-v1_5 mit dem privaten RSA-Schlüssel (n, d) auf eine Nachricht m umfasst drei Schritte:

- Mit einem Hashverfahren *Hash* wird der Hashwert

$$h = \text{Hash}(m)$$

der Nachricht m berechnet.

- Der Hashwert h wird zu einer Zahl

$$\tilde{h} < n$$

kodiert.

- Auf $\tilde{h}$ wird die private RSA-Operation angewandt. Die Signatur der Nachricht ist:

$$s = \tilde{h}^d \bmod n$$

Die Berechnung einer RSASSA-PKCS1-v1_5-Signatur lässt sich in einem Java-Programm mit einer geeigneten Instanz der Klasse `Signature` bequem durchführen. Siehe hierzu den Beispielcode in Abbildung 3.5, Zeilen 5–8.

Zur Verifizierung der RSASSA-PKCS1-v1_5-Signatur s mit dem öffentlichen RSA-Schlüssel (n, e) werden folgende Schritte ausgeführt:

- Wie bei der Signaturberechnung wird der Hashwert

$$h = \text{Hash}(m)$$

der Nachricht m berechnet und danach zu der Zahl $\tilde{h}$ kodiert.

- Auf s wird die öffentliche RSA-Operation angewendet. Genau dann, wenn das Ergebnis mit $\tilde{h}$ übereinstimmt, d. h., falls

$$\tilde{h} = s^e \bmod n$$

gilt, ist die Signatur korrekt.

Von dem in Abbildung 3.5 dargestellten Beispielcode wird zuerst der Hashwert h der Nachricht und dann das Ergebnis der Exponentiation (modulo n) des Signaturwerts s mit dem öffentlichen Exponenten e , also die Kodierung $\tilde{h}$ von h , ausgegeben. Die Ausgaben sind in Abbildung 3.6 wiedergegeben.

Offensichtlich steht der Hashwert h unverändert am Ende von $\tilde{h}$. Dabei ist h Teil der Kodierung eines Wertes vom Datentyp `DigestInfo`, der mithilfe der ASN.1 (*Abstract Syntax Notation 1*) wie folgt definiert ist:

```

DigestInfo ::= SEQUENCE {
    digestAlgorithm AlgorithmIdentifier,
    digest OCTET STRING
}

```

Hierbei sind SEQUENCE und OCTET STRING grundlegende, in ASN.1 bereits spezifizierte Datentypen (*Built-in-Types*). Der Datentyp DigestInfo besteht entsprechend der obigen Definition aus genau zwei Komponenten, einem Wert vom Typ AlgorithmIdentifier, der wiederum einen OBJECT IDENTIFIER zur Beschreibung der verwendeten Hashfunktion enthält, und einem OCTET STRING dessen Inhalt der Hashwert h ist.

In dem konkreten Beispiel aus Abbildung 3.6 besteht die Kodierung des DigestInfo-Datenwerts aus genau 35 Bytes. Die Kodierung ist eine sogenannte Tag-Length-Value-Kodierung (*TLV-Kodierung*). Die Dekodierung dieser 35 Bytes liefert:

```

30  Tag für Datentyp Sequence
21  Länge (33 Bytes) des folgenden Datenfelds:
    30  Tag für Datentyp Sequence (hier: AlgorithmIdentifier)
    09  Länge (9 Bytes) des folgenden Datenfelds:
        06  Tag für Datentyp OBJECT IDENTIFIER
        05  Länge (5 Bytes) des folgenden Datenfelds (hier ID für SHA-1):
            2B 0E 03 02 1A
        05  Tag für Datentyp NULL (hier: SHA-1 benötigt keine Parameter)
        00  Länge (0 Bytes) des Datenfelds
    04  Tag für Datentyp OCTET STRING
    20  Länge (20 Bytes) des folgenden Datenfelds (hier: Hashwert):
        AB DA 3D ... 2A EF 0A

```

Wie in dem in Abbildung 3.6 wiedergegebenen Beispiel wird bei der RSASSA-PKCS1-v1_5-Kodierung das DigestInfo-Datenfeld immer von links mit einem Byte-Array der Form

```
00 01 FF FF FF FF ... FF FF FF FF 00
```

gepadding, sodass das sich so ergebende Byte-Array die gleiche Bytelänge wie n besitzt.

```

1  byte[] m = "Ich bin's, Alice.".getBytes();
   MessageDigest sha1 = MessageDigest.getInstance("SHA1");
   System.out.println(Dump.dump(sha1.digest(m)) + "\n");

5  Signature rsaSigner = Signature.getInstance("SHA1withRSA");
   rsaSigner.initSign(privateKey);
   rsaSigner.update(m);
   byte[] s = rsaSigner.sign();

10 Cipher rsaCipher = Cipher.getInstance("RSA/ECB/NoPadding");
    rsaCipher.init(Cipher.ENCRYPT_MODE, publicKey);
    System.out.println(Dump.dump(rsaCipher.doFinal(s)));

```

Abb. 3.5 Eine Nachricht, ihr SHA-1-Hashwert und eine RSASSA-PKCS1-v1_5-Signatur der Nachricht sowie die Potenz des Signaturwerts mit dem öffentlichen Exponenten

```

000000 AB DA 3D 9C ED F9 20 59 54 15 68 68 80 12 C8 7A
000010 F0 2A EF 0A

000000 00 01 FF FF FF FF FF FF FF FF FF FF FF FF FF FF
000010 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
000020 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
000030 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
000040 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
000050 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
000060 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
000070 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
000080 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
000090 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
0000a0 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
0000b0 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
0000c0 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
0000d0 FF FF FF FF FF FF FF FF FF FF FF FF 00 30 21 30
0000e0 09 06 05 2B 0E 03 02 1A 05 00 04 14 AB DA 3D 9C
0000f0 ED F9 20 59 54 15 68 68 80 12 C8 7A F0 2A EF 0A

```

Abb. 3.6 Hashwert und RSASSA-PKCS1-v1_5-Kodierung des Hashwerts für die im byte-Array `m` aus Abbildung 3.5 gespeicherte Nachricht

